# LAPORAN TUGAS BESAR

IF4031 Arsitektur Aplikasi Terdistribusi

## Milestone 1

Sistem Terdistribusi Dasar

# Sistem Agregasi dan Distribusi Data Kebencanaan BNPB




**DikejarDeadline**

| | |
|---|---|
| Refki Alfarizi | 13523002 |
| M Hazim R Prajoda | 13523009 |
| Adhimas Aryo Bimo | 13523052 |

**PROGRAM STUDI TEKNIK INFORMATIKA**

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG


2026

# DAFTAR ISI


**1 DESKRIPSI UMUM SISTEM** **1**
- 1.1 Latar Belakang . . . 1
- 1.2 Tujuan . . . 1
- 1.3 Ruang Lingkup . . . 1
- 1.4 Komponen BNPB dan Client . . . 2
- 1.5 Istilah dan Data Kanonik . . . 2

**2 ARSITEKTUR SISTEM** **3**
- 2.1 Diagram Arsitektur . . . 3
- 2.2 Komponen dan Tanggung Jawab . . . 4
- 2.3 Alasan Pembagian Service . . . 4
- 2.4 Model Kelas Data . . . 5
- 2.5 Protokol dan Alur Komunikasi . . . 6
- 2.6 Kepemilikan Storage dan Container . . . 6
- 2.7 Alur Permintaan dan Kegagalan . . . 6

**3 TEKNOLOGI DAN ASUMSI** **8**
- 3.1 Teknologi Terpilih . . . 8
- 3.2 Alternatif dan Alasan Pemilihan . . . 8
- 3.3 Asumsi dan Batas Implementasi . . . 9
- 3.4 Konfigurasi dan Observabilitas . . . 10
- 3.5 Kontrak Mock dan Data Awal . . . 10
- 3.6 Menjalankan dan Memverifikasi Sistem . . . 11

**4 PROBLEM 1** **13**
- 4.1 Rancangan Solusi dan Mekanisme . . . 13
- 4.2 Alasan Keputusan dan Trade-off . . . 14
- 4.3 Batasan yang Disadari . . . 15
- 4.4 Bukti Penyelesaian . . . 15
- 4.5 Jawaban atas Butir “Di laporan” . . . 18

**5 PROBLEM 2** **19**
- 5.1 Rancangan Solusi dan Mekanisme . . . 19
- 5.2 Alasan Keputusan dan Trade-off . . . 20
- 5.3 Batasan yang Disadari . . . 20
- 5.4 Bukti Penyelesaian . . . 20
- 5.5 Jawaban atas Butir “Di laporan” . . . 27

**6 PROBLEM 3** **28**
- 6.1 Rancangan Solusi dan Mekanisme . . . 28

6.2 Alasan Keputusan dan Trade-off . . . . . 29
6.3 Batasan yang Disadari . . . . . 29
6.4 Bukti Penyelesaian . . . . . 30
6.5 Jawaban atas Butir “Di laporan” . . . . . 36

**7 PROBLEM 4** **37**
7.1 Rancangan Solusi dan Mekanisme . . . . . 37
7.2 Alasan Keputusan dan Trade-off . . . . . 38
7.3 Batasan yang Disadari . . . . . 39
7.4 Bukti Penyelesaian . . . . . 39
7.5 Jawaban atas Butir “Di laporan” . . . . . 42

**8 PROBLEM 5** **43**
8.1 Rancangan Solusi dan Mekanisme . . . . . 43
8.2 Alasan Keputusan dan Trade-off . . . . . 44
8.3 Batasan yang Disadari . . . . . 45
8.4 Bukti Penyelesaian . . . . . 46
8.5 Jawaban atas Butir “Di laporan” . . . . . 52

**9 KESIMPULAN** **53**

**10 PEMBAGIAN TUGAS** **54**

**DAFTAR REFERENSI** **55**

# DESKRIPSI UMUM SISTEM

## 1.1. Latar Belakang

Sistem ini dibangun untuk memenuhi tugas besar IF4031 Arsitektur Aplikasi Terdistribusi, yang mengangkat kebutuhan pertukaran data kebencanaan antarinstansi. BMKG menyediakan informasi gempa dan peringatan tsunami, sedangkan PVMBG menyediakan laporan aktivitas gunung api. BNPB berperan sebagai simpul koordinasi yang menyatukan informasi tersebut untuk Media, Tim Lapangan, dan kebutuhan internal. Karena setiap instansi memiliki format data dan mekanisme akses sendiri, pertukaran data ini membutuhkan perantara yang dapat menangani perbedaan tersebut [1,2].

Pada Milestone 1 (M1), kebutuhan tersebut diwujudkan melalui sistem agregasi dan distribusi data. Sistem mengambil data dari mock BMKG dan PVMBG, mengubahnya ke format bersama (canonical), lalu menyediakannya melalui API dan distribusi event. Selain fungsi integrasi, implementasi harus tetap dapat melayani client ketika salah satu sumber lambat atau tidak tersedia, serta membatasi data sesuai hak akses client.

## 1.2. Tujuan

Lima problem pada M1 menjadi dasar perancangan dan pengujian sistem:

1. Menyatukan data seismik dan vulkanik, termasuk menerima tambahan field sumber tanpa restart.
2. Menjaga layanan baca tetap responsif ketika PVMBG lambat atau mengalami outage.
3. Memisahkan credential antarinstansi dan menegakkan hak akses setiap client.
4. Memisahkan komponen agar satu service dapat diganti tanpa menjalankan ulang seluruh sistem, sekaligus menjaga kepemilikan storage.
5. Mendistribusikan event ke beberapa consumer yang dapat berhenti dan pulih secara independen.

Rancangan, pertimbangan, batasan, dan bukti untuk setiap tujuan dibahas pada Bab 4–8.

## 1.3. Ruang Lingkup

Implementasi M1 menggunakan mock BMKG dan PVMBG sebagai sumber data, dengan BNPB sebagai node koordinasi. Data yang ditangani terbatas pada bahaya seismik dan vulkanik. Kedua mock menyediakan data sintetis dan kontrol untuk menguji perubahan schema, delay, serta outage.

Ketiga client hanya membaca data. Pelaporan dari lapangan, kebakaran hutan, serta jenis bahaya lain belum termasuk cakupan M1 [1,2]. Dua consumer memperagakan pembaruan dashboard dan notifikasi, dengan notifikasi dicatat di storage lokal sebagai simulasi.

## 1.4. Komponen BNPB dan Client

Spesifikasi membagi sistem BNPB menjadi enam komponen [1]. Tabel 1.1 merangkum peran setiap komponen. Mock BMKG dan PVMBG berada di sisi sumber data dan berkomunikasi dengan sistem ini melalui HTTP.

| Komponen BNPB | Peran dalam sistem |
|---|---|
| Gateway / Aggregator | Mengambil data kedua sumber, melakukan validasi dan pemetaan, lalu menyimpan hasil serta menyiapkan event untuk diterbitkan. |
| Canonical Store | Menyimpan data dalam format bersama, riwayat perubahan, status sumber, dan antrean event yang akan dikirim. Storage ini dikelola Aggregator. |
| Client-Facing API | Menyediakan jalur baca bagi client dan menegakkan pembatasan data sesuai token. |
| Auth / Token Service | Memeriksa identitas client, menerbitkan access token, dan mengelola refresh token. |
| Message Broker | Menyimpan event dan menyalurkannya ke consumer dengan posisi baca yang independen. |
| Event Consumer ($\geq 2$) | Mengolah event untuk kebutuhan masing-masing aplikasi. Implementasi menggunakan Dashboard dan Notifier. |

Tabel 1.1: Enam komponen BNPB sesuai spesifikasi M1.

Client mengakses data melalui Client-Facing API, dengan identitas yang dibedakan oleh Auth. Hak aksesnya mengikuti kontrak pada Tabel 1.2; pembatasan tersebut diterapkan di server.

| Client | Hak akses |
|---|---|
| Media / Pers | Ringkasan kejadian tanpa koordinat presisi, identifier sumber, atau atribut raw. |
| Tim Lapangan | Seluruh field, termasuk koordinat dan atribut raw, dengan access token berumur pendek yang dapat diperbarui melalui refresh. |
| BNPB Pusat / Internal | Seluruh field sebagai baseline hak akses tertinggi. |

Tabel 1.2: Tiga identitas client dan batas data yang dapat dibaca.

## 1.5. Istilah dan Data Kanonik

Rincian pemetaan dibahas pada Bab 4. Istilah *kanonik* merujuk pada format bersama untuk mewakili kejadian dari kedua sumber. Format ini bernama `HazardEvent`, dengan identitas, jenis bahaya, severity, lokasi, waktu, dan atribut sumber.

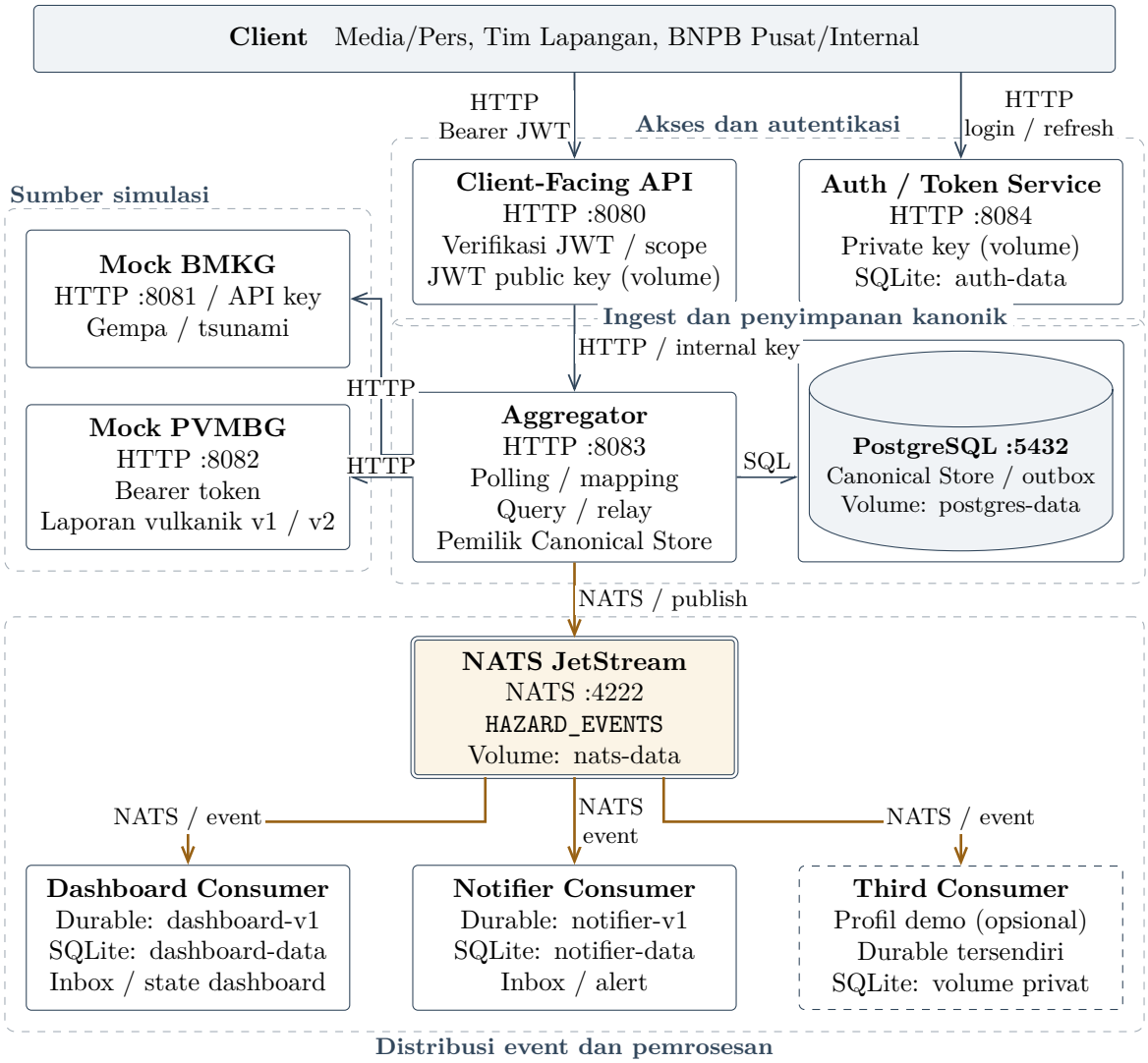
BMKG menyediakan `SeismicEvent` dan `TsunamiWarning`, yang dihubungkan melalui `related_event_id`. PVMBG menyediakan `VolcanicReport`. Aggregator memetakan kedua sumber ke HazardEvent, dengan rincian khusus sumber dan field baru, seperti `confidence_level`, disimpan dalam `attributes`.

# ARSITEKTUR SISTEM

## 2.1. Diagram Arsitektur

Sistem memiliki dua jalur utama: penyediaan data melalui API dan distribusi perubahan melalui event. Keduanya berawal dari data yang diambil Aggregator, tetapi berjalan terpisah setelah data tersimpan. Dengan pemisahan ini, client dapat membaca data terakhir tanpa menunggu polling, sementara setiap consumer mengolah event sesuai kecepatannya sendiri.

Gambar 2.1 merangkum service, storage, dan jalur komunikasinya.




Gambar 2.1: Kotak menunjukkan batas container; bingkai putus-putus mengelompokkan tanggung jawab. Silinder menunjukkan database dan kotak ganda menunjukkan broker. Panah hitam menunjukkan arah request; panah berwarna menunjukkan pengiriman event ke consumer.

## 2.2. Komponen dan Tanggung Jawab

Setiap service mengelola satu bagian dari alur data, sehingga perubahan dan gangguan dapat dibatasi pada bagian tersebut. Dua mock mewakili instansi sumber: BMKG menyediakan gempa dan warning, sedangkan PVMBG menyediakan laporan vulkanik. Keduanya bersifat pasif dan hanya mengirim data sebagai respons terhadap polling. Endpoint admin mendukung simulasi perubahan schema, delay, dan outage.

Aggregator menjadi penghubung antara kontrak sumber dan kontrak BNPB. Service ini memvalidasi payload, menghubungkan warning dengan gempa, lalu memetakan hasilnya ke HazardEvent. Aggregator juga mengelola Canonical Store dan menyediakan API internal agar service lain dapat meminta data tanpa bergantung pada struktur tabelnya.

Akses client ditangani oleh Client-Facing API dan Auth. Auth memeriksa identitas dan menerbitkan token dengan scope yang sesuai. Setelah itu, Client API memverifikasi token dan menyaring field yang boleh dibaca. Dengan demikian, pengambilan data dari sumber tetap terpisah dari aturan penyajiannya kepada client.

Untuk distribusi perubahan, NATS JetStream menyimpan event yang diterbitkan Aggregator. Dashboard Consumer mengolahnya menjadi state dashboard, sedangkan Notifier Consumer mencatat alert. Setiap consumer memiliki posisi baca dan database sendiri, sehingga berhentinya satu consumer tidak menghentikan consumer lain.

## 2.3. Alasan Pembagian Service

Pembagian service mengikuti perbedaan tanggung jawab, kebutuhan akses, dan kepemilikan data. Bagian yang mengelola data yang sama tetap digabung, sedangkan bagian yang perlu berubah atau pulih secara independen dipisahkan. Pertimbangan ini menghasilkan batas berikut.

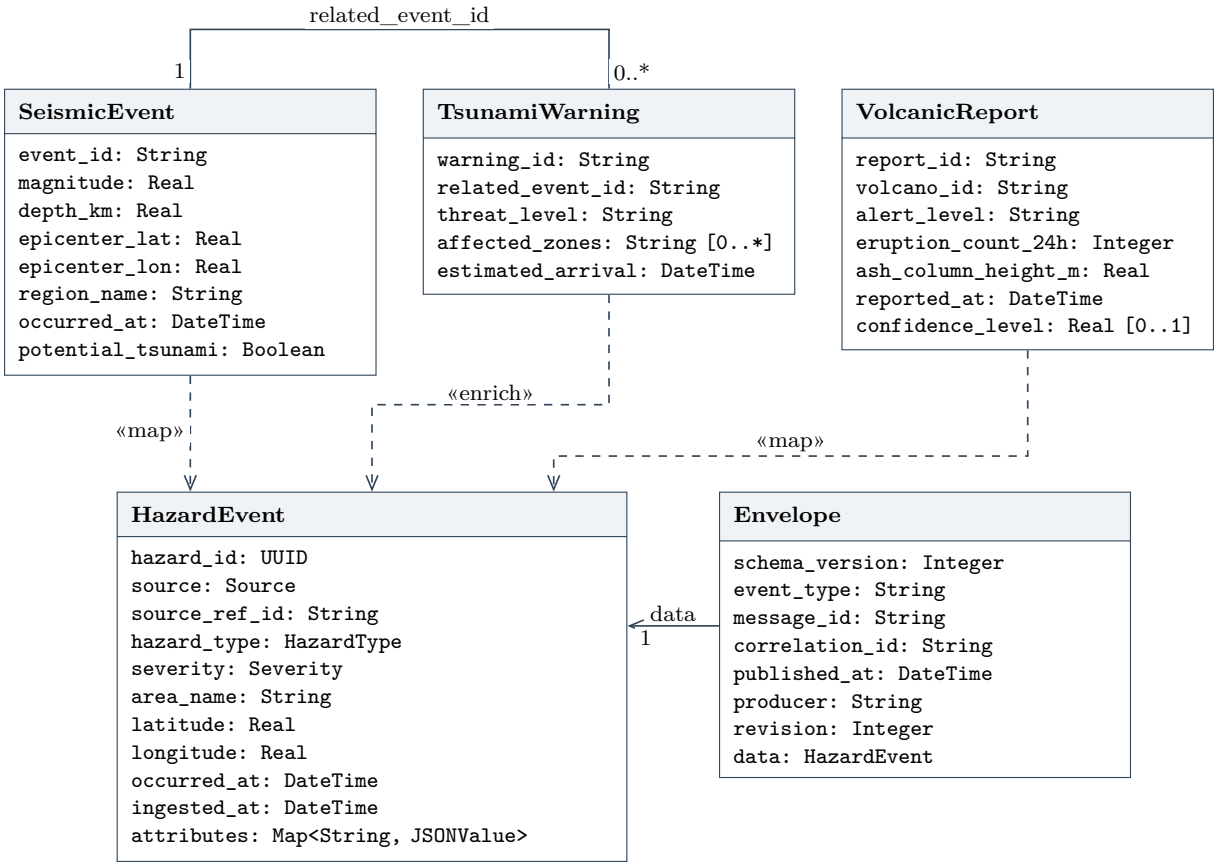
Polling, mapping, query internal, dan relay ditempatkan dalam Aggregator karena semuanya menggunakan Canonical Store yang sama. Loop dan pool koneksi dipisahkan untuk mengisolasi pekerjaan di dalam service tersebut. Alternatifnya, ingest tiap instansi dapat menjadi service sendiri, tetapi perlu antarmuka serta koordinasi storage tambahan. Pilihan satu Aggregator lebih sederhana untuk M1, dengan konsekuensi semua pekerjaan ingest berhenti jika containernya mati.

Client API dipisahkan dari Aggregator agar perubahan hak akses dan format respons tidak mengharuskan rebuild jalur ingest. Auth menjadi service tersendiri untuk menjaga private key penandatanganan token, sementara API cukup memegang public key. Verifikasi JWT dilakukan secara lokal di API, sehingga request dengan token yang masih valid tetap dapat dilayani ketika Auth berhenti. Login dan refresh tetap membutuhkan Auth.

Dashboard dan Notifier dipisahkan karena masing-masing menghasilkan state aplikasi yang berbeda. Container, database, dan durable consumer tersendiri memungkinkan keduanya dipulihkan secara independen. Third Consumer digunakan untuk menguji penambahan aplikasi pembaca: consumer ini memakai binary Notifier, tetapi memiliki durable dan volume sendiri. Dengan kontrak event yang sama, producer tidak perlu diubah ketika pembaca baru ditambahkan.

## 2.4. Model Kelas Data

Untuk menyatukan data kedua instansi, Aggregator menggunakan HazardEvent sebagai kontrak bersama. Gambar 2.2 memperlihatkan model data sumber, hasil pemetaan ke HazardEvent, serta envelope yang membawanya ke broker. Gempa dan warning dihubungkan melalui identifier sumber; informasi warning memperkaya kejadian gempa yang terkait. Laporan vulkanik dipetakan melalui jalur tersendiri ke format kanonik yang sama.




Gambar 2.2: Model data dan pemetaannya. Satu warning merujuk satu gempa; satu gempa dapat mempunyai banyak warning. Field `confidence_level` bersifat opsional dan muncul pada PVMBG v2. Envelope membawa satu snapshot HazardEvent.

Beberapa field HazardEvent menggunakan nilai yang dibatasi agar client dapat menafsirkan data secara konsisten. Field `source` membedakan BMKG dan PVMBG, sedangkan `hazard_type` membedakan SEISMIC dan VOLCANIC. Tingkat keparahan dinyatakan melalui `severity`, dengan pilihan NORMAL, WASPADA, SIAGA, dan AWAS. Informasi khusus sumber disimpan dalam `attributes`. Pada diagram, `JSONValue` mewakili nilai JSON, dan `DateTime` mewakili timestamp yang diserialisasikan sebagai string.

HazardEvent mewakili kejadian, sementara envelope memuat metadata untuk pengiriman, seperti identitas pesan, revisi, dan correlation ID. Pemisahan ini memungkinkan consumer mengenali perubahan dan pengiriman ulang tanpa mengubah kontrak kejadian. Warning yang gempa terkaitnya belum tersedia tetap menunggu kecocokan pada polling berikutnya; warning tersebut tidak dipetakan menjadi hazard mandiri.

## 2.5. Protokol dan Alur Komunikasi

Komunikasi sistem mengikuti dua alur: pembaruan data dari sumber dan pembacaan data oleh client. Alur pembaruan juga menghasilkan event untuk consumer. Urutannya adalah sebagai berikut.

**Pembaruan data dan distribusi event.** Aggregator melakukan polling HTTP dengan credential milik masing-masing sumber. Untuk BMKG, endpoint `/seismic-events` dan `/tsunami-warnings` diambil secara concurrent; PVMBG diambil melalui `/volcanic-reports` pada loop independen. Setelah validasi dan pemetaan, Aggregator menyimpan data terbaru, revisi, serta pesan outbox dalam satu transaksi PostgreSQL.

Outbox merupakan catatan event yang perlu dikirim. Relay membacanya dan menerbitkan envelope melalui protokol NATS. Setelah menerima PubAck dari JetStream, relay menandai pesan telah terpublikasi. Dashboard dan Notifier kemudian meminta pesan dari stream melalui durable pull consumer masing-masing, mengolahnya ke database lokal, lalu mengirim ACK. Mekanisme outbox dan pengiriman ulang dijelaskan pada Bab 8.

**Pembacaan oleh client.** Client terlebih dahulu login ke Auth melalui HTTP untuk memperoleh token. Pada request data berikutnya, client mengirim Bearer JWT ke Client API. API memverifikasi token secara lokal, memeriksa scope, lalu meminta data ke API internal Aggregator menggunakan key tersendiri. Aggregator membaca Canonical Store, dan Client API mengembalikan field yang diizinkan untuk identitas tersebut. Jalur ini tidak menunggu request baru ke BMKG atau PVMBG.

## 2.6. Kepemilikan Storage dan Container

Setiap service mengakses data persisten melalui storage yang menjadi tanggung jawabnya. Canonical Store berada di PostgreSQL dan hanya diakses langsung oleh Aggregator. Store ini memuat data terbaru pada `hazard_events`, riwayat pada `hazard_versions`, status sumber pada `source_state`, serta pesan pada `outbox`. Service lain menggunakan API atau kontrak event untuk memperoleh data.

Auth menyimpan state refresh token dalam SQLite privat. Dashboard dan Notifier juga memiliki SQLite masing-masing untuk mencatat pesan yang sudah diproses dan hasil olahannya. Sementara itu, NATS menyimpan stream pada volume broker. Seluruh storage tersebut menggunakan volume persisten agar penggantian container tidak menghapus state. Sembilan service utama berjalan dalam container terpisah melalui Compose; Third Consumer ditambahkan hanya untuk pengujian melalui profil demo.

## 2.7. Alur Permintaan dan Kegagalan

Dalam kondisi normal, request client melewati Client API dan Aggregator untuk membaca data yang sudah tersimpan. Polling berjalan di background untuk memperbarui data tersebut. Karena kedua pekerjaan terpisah, delay PVMBG tidak membuat request client menunggu respons PVMBG dan tidak menghalangi polling BMKG.

Saat PVMBG mengalami outage, Aggregator tetap menyediakan record terakhir yang berhasil disimpan, dengan penanda *stale* untuk menunjukkan bahwa data belum

diperbarui. Jika belum ada record yang dapat dilayani, request khusus sumber tersebut memperoleh status unavailable. Ketika sumber pulih, polling berikutnya memperbarui data dan status freshness tanpa perlu restart. Perilaku serta batas ketersediaannya diuji pada Bab 5.

Pada jalur event, kegagalan broker membuat pesan tetap menunggu di outbox untuk dicoba kembali. Jika hanya consumer yang berhenti, broker tetap menyimpan event dan consumer lain tetap dapat membacanya. Consumer yang pulih melanjutkan pembacaan menggunakan posisi durable selama pesan yang dibutuhkan masih berada dalam retention. Pemisahan ini tidak menyediakan failover untuk Aggregator, PostgreSQL, atau broker yang masing-masing masih menggunakan satu instance.

### 2.7.1 Batas Jaringan

Jaringan Compose membatasi jalur komunikasi sesuai kebutuhan service. Pemisahan terpenting adalah Canonical Store: hanya Aggregator yang perlu terhubung langsung ke PostgreSQL. Keanggotaan jaringan dirangkum pada Tabel 2.1.

| Jaringan | Anggota | Tujuan |
|---|---|---|
| `source-net` | Mock BMKG, mock PVMBG, Aggregator | Polling dan autentikasi ke sumber. |
| `api-net` | Client API, Auth, Aggregator | Login/refresh dari client dan pembacaan melalui API internal. |
| `store-net` | Aggregator, PostgreSQL | Akses SQL privat ke Canonical Store. |
| `event-net` | Aggregator, NATS, seluruh consumer | Publish, pull, dan ACK event. |

Tabel 2.1: Pemisahan jaringan pada Docker Compose.

Jaringan `store-net` dan `event-net` bersifat internal; PostgreSQL dan NATS tidak memublikasikan port ke host. Port aplikasi yang diperlukan untuk demo terikat ke `127.0.0.1`. Konfigurasi ini membatasi akses dari luar host, tetapi komunikasi internal masih menggunakan HTTP tanpa TLS.

# TEKNOLOGI DAN ASUMSI

## 3.1. Teknologi Terpilih

Pemilihan teknologi mengikuti kebutuhan M1: menjalankan layanan HTTP secara concurrent, menyimpan data yang schema-nya dapat bertambah, dan mendistribusikan event yang tetap tersedia saat consumer berhenti. Tabel 3.1 menghubungkan teknologi yang digunakan dengan peran setiap komponen. Pertimbangan terhadap alternatifnya dijelaskan setelah tabel.

| Teknologi | Komponen | Kebutuhan yang dipenuhi |
|---|---|---|
| Go | Kedua mock, Aggregator, Client API, Auth, dan consumer | Handler HTTP serta goroutine untuk pekerjaan concurrent; context untuk deadline dan pembatalan [3,4]. |
| PostgreSQL + JSONB | Canonical Store milik Aggregator | Transaksi data, history, dan outbox; atribut dinamis dapat bertambah tanpa kolom baru [5]. |
| NATS JetStream | Message Broker | Stream persisten dan posisi durable terpisah agar consumer dapat mengejar event setelah pulih [6,7]. |
| SQLite | State privat Auth dan setiap consumer | Menyimpan sesi refresh, inbox, serta hasil olahan secara persisten tanpa server database tambahan. |
| Docker Compose | Seluruh stack | Mengatur container, jaringan, dan volume pada satu host, serta menjalankan ulang satu service secara terpisah [9]. |

Tabel 3.1: Teknologi, komponen pengguna, dan kaitannya dengan kebutuhan sistem.

Modul aplikasi menggunakan minimum Go 1.23, dengan `pgx/v5` untuk PostgreSQL dan `nats.go` untuk broker. Compose memakai image PostgreSQL 16 dan NATS 2.10. Tag tersebut belum dikunci dengan image digest, sehingga versi patch pada lingkungan pengujian dicatat bersama hasilnya. Fixture kontrak diperiksa menggunakan JSON Schema Draft 2020-12.

## 3.2. Alternatif dan Alasan Pemilihan

**Go.** Pekerjaan HTTP dan polling membutuhkan konkurensi serta pengendalian timeout. Go menyediakan goroutine, `net/http`, dan `context` dalam ekosistem yang sama, sehingga kebutuhan tersebut dapat diterapkan tanpa framework aplikasi tambahan. Service juga dibangun menjadi binary tersendiri untuk container. Python atau Java tetap dapat memenuhi kebutuhan ini; Go dipilih untuk menyederhanakan implementasi concurrent dan deployment kelompok.

**PostgreSQL dengan JSONB.** Data kanonik membutuhkan constraint identitas dan transaksi, sedangkan atribut sumber dapat berubah. Menggunakan kolom tetap un-

tuk seluruh atribut akan membutuhkan migrasi ketika field baru ditambahkan. Document store seperti MongoDB menawarkan fleksibilitas dokumen, tetapi konsistensi data terbaru, history, dan outbox tetap harus dirancang. PostgreSQL dengan JSONB memenuhi kedua kebutuhan dalam satu store. Konsekuensinya, atribut dinamis tetap perlu validasi dan query tertentu dapat membutuhkan index tambahan; perbandingan storage dibahas pada Bab 7.

**NATS JetStream.** Prioritas broker M1 adalah persistence, replay, dan durable consumer dengan setup yang sederhana pada satu host. JetStream tersedia di dalam executable `nats-server`, sehingga konfigurasi lokal cukup menjalankan satu container dengan volume data. Bentuk ini ringan dari sisi operasional untuk demo. NATS Core lebih sederhana, tetapi tidak menyimpan backlog bagi consumer yang offline [6,7].

RabbitMQ menyediakan routing melalui exchange dan binding [12], sedangkan Kafka menyediakan log terpartisi yang dapat dibaca ulang [13]. Keduanya dapat mendukung distribusi event. Namun, M1 berfokus pada fan-out dan recovery consumer pada satu host, tanpa kebutuhan routing yang beragam atau pengaturan partisi untuk skala besar. JetStream dipilih karena kebutuhan tersebut cukup dipenuhi oleh stream dan durable dalam satu server NATS. Konsekuensinya, retention, redelivery, dan idempotensi tetap harus diatur; deployment satu node juga belum memberikan failover.

**SQLite privat.** Auth dan consumer membutuhkan state yang bertahan saat restart, tetapi masing-masing hanya berjalan sebagai satu instance. SQLite memberi transaksi lokal tanpa menambah server database. Penyimpanan memory kehilangan state ketika proses berhenti, sedangkan database server bersama menambah dependency serta pengaturan akses. SQLite sesuai untuk deployment ini; jika beberapa replica harus mengelola state yang sama, pola penyimpanannya perlu ditinjau kembali.

**Docker Compose.** Eksperimen M1 dijalankan pada satu host, sehingga Compose sudah cukup untuk mendefinisikan service, network, dan volume. Kubernetes menjadi alternatif ketika pengelolaan beberapa node dan pemulihan workload dibutuhkan, tetapi menambah konfigurasi yang belum diperlukan pada demo ini. Konsekuensinya, deployment Compose tidak memberi failover lintas mesin.

## 3.3. Asumsi dan Batas Implementasi

Rancangan menggunakan asumsi berikut untuk menetapkan cakupan data dan deployment. Asumsi tersebut juga menentukan batas interpretasi hasil pengujian.

1. Data berasal dari fixture sintetis. Nama dan koordinat gunung pada referensi Aggregator menggunakan simulasi Tamriel dan tidak mewakili lokasi gunung sebenarnya.
2. Sumber tetap menyediakan endpoint dan field wajib sesuai kontrak. Perubahan aditif diterima otomatis, sedangkan penghapusan, rename, atau perubahan tipe field wajib membutuhkan pembaruan kontrak.
3. Ingest menggunakan full snapshot agar warning terlambat dan perubahan pada ID lama dapat dibaca kembali. Pendekatan ini mengasumsikan snapshot masih berada dalam batas ukuran dan waktu pemrosesan; biaya baca bertambah ketika riwayat sumber membesar.
4. Setiap komponen dijalankan sebagai satu instance pada satu host. Replikasi, failover lintas host, dan koordinasi beberapa worker ingest belum diterapkan.
5. Client mengikuti alur login, penggunaan access token, dan refresh. Expiry token

serta metadata freshness bergantung pada clock host, sehingga waktu host perlu dijaga tetap sinkron.

## 3.4. Konfigurasi dan Observabilitas

Konfigurasi dipisahkan dari source agar credential dan parameter eksperimen dapat diganti tanpa mengubah kode. Repository menyediakan `.env.example` sebagai daftar variabel, dengan secret aktif diisi pada `.env` lokal yang tidak masuk Git. Setiap service menerima konfigurasi sesuai perannya; key BMKG, token PVMBG, key internal Aggregator, token NATS, dan secret tiap client menggunakan nilai berbeda.

Tabel 3.2 merangkum nilai awal Aggregator. Interval polling mengatur jeda pembaruan, timeout membatasi waktu pekerjaan, sedangkan pool dan batas antrean mengendalikan resource. Nilai-nilai ini merupakan konfigurasi awal yang digunakan sebagai dasar pengujian, bukan hasil pencarian kapasitas maksimum.

| Parameter Aggregator | Default implementasi |
|---|---|
| Interval poll / stale threshold | 5 s / 15 s |
| Timeout HTTP BMKG / PVMBG | 1 s / 4 s |
| Batas respons sumber / envelope event | 1 MiB / 64 KiB |
| Pool baca / tulis BMKG / tulis PVMBG / relay | 10 / 2 / 2 / 2 koneksi |
| Deadline query baca / transaksi tulis | 200 ms / 2 s |
| Admission API internal | 128 request data aktif |
| Relay poll / batch / pending maksimum | 500 ms / 20 pesan / 10.000 pesan |
| Timeout publish / rentang retry | 2 s / 1–30 s dengan jitter |

Tabel 3.2: Konfigurasi awal untuk polling, pembacaan, dan pengiriman event.

Untuk pemantauan, endpoint `/health` menunjukkan proses masih hidup, sedangkan `/ready` memeriksa dependency sesuai peran service. Aggregator tetap siap melayani baca ketika sumber atau broker gagal selama database dapat dibaca. Log terstruktur mencatat operasi, correlation ID, durasi, dan status agar satu alur dapat ditelusuri tanpa mencetak credential atau payload mentah. Saat shutdown, context dibatalkan dan worker ditunggu sebelum resource ditutup.

## 3.5. Kontrak Mock dan Data Awal

Kedua mock merepresentasikan sumber yang berbeda, termasuk kontrak autentikasi dan karakteristik delay. BMKG melayani `/seismic-events` serta `/tsunami-warnings` dengan header `X-BMKG-Key` dan delay default 100 ms. PVMBG melayani `/volcanic-reports` dengan Bearer token dan delay acak 500–3.000 ms. Credential diperiksa sebelum delay diterapkan.

Untuk eksperimen, PVMBG menyediakan endpoint admin `/admin/schema-version`, `/admin/outage`, dan `/admin/delay`, dengan credential yang sama seperti endpoint datanya. Toggle schema v2 menambahkan `confidence_level` pada seluruh

laporan. Outage membuat endpoint data menghasilkan 503, sementara endpoint admin dan liveness tetap dapat diakses untuk memulihkan sumber.

Secara default, kedua mock memulai dengan 20 record historis dan menambah record setiap 10 detik. ID memuat run ID agar data dari restart mock tidak menimpa riwayat sebelumnya. Aggregator mulai melakukan polling saat startup, kemudian mengulangnya setiap 5 detik. Setiap sumber hanya memiliki satu siklus aktif; tick yang datang saat siklus masih berjalan dilewati.

## 3.6. Menjalankan dan Memverifikasi Sistem

Sistem dijalankan dari root repository menggunakan Docker Compose. Persiapan meliputi pengisian credential lokal dan pembuatan pasangan RSA untuk JWT. Ringkasan berikut mengikuti petunjuk pada `scripts/README.md`; README root menjadi titik masuk dokumentasi repository.

### 3.6.1 Urutan Menjalankan Demo

Salin `.env.example` menjadi `.env`, lalu isi secret berbeda untuk setiap domain dan client serta tentukan `COMPOSE_PROJECT_NAME`. Muat konfigurasi tersebut ke environment shell agar command HTTP dan script load test dapat menggunakan credential yang sama dengan Compose. Untuk setup pertama, buat pasangan key RSA dan muat ke volume private/public yang terpisah:

```
cp .env.example .env
# Isi konfigurasi dan credential lokal pada .env.
set -a
. ./.env
set +a
mkdir -p .secrets
chmod 700 .secrets
(umask 077; openssl genpkey -algorithm RSA \
  -pkeyopt rsa_keygen_bits:2048 -out .secrets/jwt-private.pem)
openssl pkey -in .secrets/jwt-private.pem -pubout \
  -out .secrets/jwt-public.pem
python3 scripts/setup-jwt-keys.py
docker compose config --quiet
docker compose up --build -d --wait
```

Pada startup berikutnya, gunakan pasangan key yang sudah ada. Tangkapan layar laporan menggunakan port 8280–8284 untuk API, BMKG, PVMBG, Aggregator, dan Auth secara berurutan. Untuk mengulang command tersebut, sesuaikan port listener dan base URL antarservice pada `.env`. Setelah service siap, client login ke Auth untuk memperoleh token sebelum membaca API. Consumer ketiga dapat ditambahkan melalui profil demo:

```
docker compose --profile demo up -d --no-deps third-consumer
```

Trial integrasi memeriksa schema, token, outage, penggantian container, dan recovery consumer:

```
python3 scripts/verify-integration.py --scenario all \
  --output docs/evidence/integration-trial.json
```

Trial ini mengubah kontrol sumber dan menghentikan beberapa service sementara, sehingga dijalankan pada stack khusus eksperimen. Hasil setiap problem dibahas pada Bab 4–8.

### 3.6.2 Lingkungan dan Cakupan Pengujian

Bukti laporan berasal dari pengujian lintas service pada 5 Oktober 2026 dan tangkapan command pada 7 Oktober 2026 [10]. Trial 5 Oktober menggunakan commit `071b32d`, dengan WSL2 Linux, 12 CPU logical, Docker 27.0.3, Compose 2.28.1, PostgreSQL 16.14, dan NATS 2.10.29. Image aplikasi dibangun dengan Go 1.23. Port host API hingga Auth pada eksperimen menggunakan 8280–8284.

Pengujian komponen melengkapi trial tersebut. Pada commit `19beead`, tes unit/HTTP, race detector, build, vet, smoke mock, serta tes Store dan publisher lulus. Tes native menggunakan Go 1.26.4, PostgreSQL 18.6, dan NATS 2.10.24. Selain itu, uji regresi availability pada commit `e94e03b` memeriksa pembacaan record valid dari ingest parsial. Perbedaan versi dan commit dicatat agar hasil komponen dapat dibedakan dari hasil deployment Compose. Seluruh hasil menggambarkan profil yang diuji; keterbatasan pada snapshot besar dibahas pada P1.

# PROBLEM 1

# ARSITEKTUR & INTEROPERABILITAS DATA

## 4.1. Rancangan Solusi dan Mekanisme

Problem 1 berfokus pada penyatuan format data serta penanganan penambahan field dari sumber [1]. BMKG menyediakan gempa dan warning tsunami pada endpoint terpisah, sedangkan PVMBG menyediakan laporan gunung api tanpa koordinat langsung. Untuk mengatasi perbedaan tersebut, Aggregator memetakan data ke HazardEvent dengan aturan per sumber. Field wajib tetap divalidasi, sementara informasi tambahan dipertahankan sebagai atribut.

### 4.1.1 Model kanonik dan pemetaan sumber

HazardEvent memberi client bentuk data yang konsisten untuk kedua jenis bahaya. Sebelas field-nya mencakup identitas, sumber, tipe, severity, lokasi, waktu, dan `attributes`, seperti pada model data di Bab 2. Mapper membaca field sumber sebagai `json.RawMessage` agar field yang belum dikenal tetap dapat dipertahankan. Field wajib kemudian divalidasi, dan nilai angka dinamis menggunakan `json.Number` agar presisinya terjaga.

Untuk BMKG, `event_id` menjadi `source_ref_id`, region dan koordinat episentrum dipertahankan, serta jenis bahaya diisi `SEISMIC`. Warning divalidasi terpisah, lalu dikelompokkan melalui `related_event_id` untuk dihubungkan dengan gempa. Jika ada warning rusak, gempa valid tetap diproses, tetapi siklus polling ditandai degraded karena belum berhasil penuh.

Severity gempa mengikuti warning jika `potential_tsunami=true` dan warning terkait tersedia. Apabila ada beberapa warning, tingkat tertinggi dipilih dan seluruh warning valid disimpan terurut dalam `attributes`. Tanpa warning, magnitude di bawah 5,0 menjadi NORMAL, 5,0 hingga kurang dari 6,5 menjadi WASPADA, dan mulai 6,5 menjadi SIAGA.

Untuk PVMBG, `report_id` menjadi source reference dan `reported_at` menjadi waktu kejadian. Alert level dipetakan ke enum severity kanonik, sedangkan area dan koordinat diambil dari tabel referensi sintetis Tamriel berdasarkan `volcano_id`. Laporan dengan ID gunung yang tidak dikenal ditolak karena lokasinya tidak dapat ditentukan secara valid.

Identitas hazard perlu tetap sama ketika sumber memperbarui record lama. Karena itu, Aggregator mengidentifikasi kejadian dengan menggabungkan `source` dan `source_ref_id`. Record pertama memperoleh UUIDv4; perubahan berikutnya mempertahankan UUID dan menambah revisi. Polling full snapshot memungkinkan warning yang baru tersedia ikut memperbarui hazard lama. Jika isi record tidak berubah, polling hanya memperbarui freshness sumber tanpa menambah revisi atau pesan outbox.

### 4.1.2 Ringkasan pemetaan sumber

Tabel berikut memperlihatkan padanan field utama dari setiap sumber. Informasi yang tidak mempunyai field kanonik tersendiri ditempatkan dalam `attributes`, sehingga rincian sumber tetap tersimpan.

| Field kanonik | BMKG | PVMBG |
|---|---|---|
| `source_ref_id` | `event_id` | `report_id` |
| `hazard_type` | SEISMIC | VOLCANIC |
| `area_name` | `region_name` | Nama gunung hasil lookup `volcano_id` |
| `latitude`, `longitude` | `epicenter_lat`, `epicenter_lon` | Koordinat dari fixture referensi gunung |
| `occurred_at` | `occurred_at` | `reported_at` |
| `severity` | Batas magnitude atau warning terkait | Alert level ke enum kanonik |
| `attributes` | Magnitude, kedalaman, potensi tsunami, warning, field tambahan | ID gunung, erupsi, tinggi abu, confidence jika tersedia, field tambahan |

ID hazard dibuat pada ingest pertama, sedangkan waktu ingest mencatat saat data diterima. Timestamp dinormalisasi ke UTC; bentuk serialisasi `Z` dan `+00:00` menunjukkan timezone yang sama.

### 4.1.3 Penanganan perubahan schema

Perubahan yang diuji adalah penambahan `confidence_level` pada PVMBG v2. Karena field tersebut belum mempunyai padanan kanonik, mapper meneruskannya ke `attributes`, yang disimpan sebagai JSONB. Dengan demikian, field baru dapat masuk tanpa perubahan kolom atau restart. Jika nama field sumber bertabrakan dengan atribut hasil turunan, nilainya dipertahankan dalam `source_extras`.

Fleksibilitas tersebut tetap dibatasi oleh kontrak wajib. Record dengan field wajib yang hilang, tipe atau enum salah, duplicate key, maupun JSON invalid ditolak. Record valid tetap diproses, tetapi siklus degraded tidak memperbarui waktu sukses terakhir sumber.

## 4.2. Alasan Keputusan dan Trade-off

Pilihan parsing ditentukan oleh kebutuhan menerima field baru tanpa kehilangan data. Decode langsung ke struct dengan field tetap lebih sederhana, tetapi field tambahan dapat terabaikan. RawMessage dan validasi eksplisit dipilih agar kontrak wajib tetap diperiksa sekaligus mempertahankan field yang belum dikenal [4]. Konsekuensinya, mapper perlu menangani validasi dan benturan nama atribut secara eksplisit.

Untuk polling, cursor incremental dapat mengurangi volume baca, tetapi belum tentu menangkap perubahan pada ID lama atau warning yang datang belakangan. Full snapshot dipilih agar kedua kasus dapat dibaca kembali pada siklus berikutnya. Trade-offnya adalah ukuran dan biaya pemrosesan snapshot terus bertambah. JSONB membantu

penyimpanan atribut dinamis, sementara validasi domain tetap menjadi tanggung jawab mapper [5].

## 4.3. Batasan yang Disadari

Dukungan perubahan schema dibatasi pada penambahan field. Field baru disimpan tanpa otomatis masuk ke ringkasan atau mengubah severity, karena keduanya mengikuti aturan domain dan hak akses. Rename, penghapusan, atau perubahan tipe field wajib membutuhkan pembaruan kontrak; mapper tidak menebak makna field pengganti agar hasil kanonik tidak keliru.

Pemetaan lokasi PVMBG menggunakan fixture Tamriel untuk simulasi dan belum menjadi referensi geografis nyata. Jalur warning terlambat sudah ditangani oleh rancangan full snapshot, tetapi belum diuji sebagai satu skenario acceptance tersendiri dari poller sampai revisi PostgreSQL.

Polling masih menggunakan full snapshot tanpa pagination atau batch ingest. Pilihan ini memudahkan korelasi dan pembaruan record lama, tetapi biaya pemrosesan bertambah saat data sumber membesar. Ingest pada snapshot besar masih dapat gagal, sehingga dukungan perubahan schema pada volume tersebut belum dapat dinyatakan andal.

## 4.4. Bukti Penyelesaian

Pengujian P1 memeriksa dua hal: ketepatan hasil pemetaan dan kelanjutan ingest ketika field baru ditambahkan. Tes komponen memeriksa aturan mapper serta penyimpanan, sedangkan trial integrasi menunjukkan perilakunya pada stack yang berjalan.

Tes mapper lulus untuk batas magnitude, prioritas dan korelasi warning, preservasi field dinamis, bentuk PVMBG v1/v2, serta penolakan enum/ID/duplicate key invalid. Tes poller lulus untuk warning rusak yang terkait maupun tidak terkait: gempa valid tetap diteruskan dan siklus ditandai degraded.

Smoke HTTP pada commit `19beead` memeriksa seed 20 record, penolakan akses tanpa credential dengan 401, perubahan PVMBG v1/v2, serta respons 503 saat outage. ID laporan tetap sama setelah toggle. Pada pengujian PostgreSQL, ID hazard juga tetap, revisi dan outbox tidak bertambah untuk payload identik, serta snapshot sebelum dan sesudah field baru tetap dapat dibaca.

### 4.4.1 Bukti Integrasi

Pengujian integrasi membandingkan hasil pemetaan dengan kontrak HazardEvent, lalu memeriksa perubahan schema melalui riwayat revisi dan kondisi container [10].

Pada contoh BMKG, magnitude 4,2 tanpa potensi tsunami menghasilkan severity NORMAL, dengan region dan koordinat yang sama seperti sumber. Contoh PVMBG v1 dengan alert Normal menghasilkan VOLCANIC/NORMAL, dan ID Red Mountain dipetakan ke koordinat sintetis $(-7, 108)$. Source reference tetap menunjuk record asal. Validasi kontrak lulus untuk delapan contoh HazardEvent dari trial integrasi dan pemeriksaan melalui command.

Pengujian perubahan schema dimulai dengan mencatat kondisi container dan daftar migrasi. Selanjutnya, `curl` mengirim body `{"version":2}` ke `POST /admin/schema-version` dengan token PVMBG. Respons `version: 2` menunjukkan bahwa perubahan telah diterapkan pada mock. Jeda 10 detik memberi kesempatan Aggregator membaca schema baru melalui polling.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'sort_by(.Name) | map({name: .Name, id: .Id,
>       started_at: .State.StartedAt, restarts: .RestartCount})' >
/tmp/report-session/schema-before.json
> docker compose exec -T aggregator /aggregator store-audit \
>   | jq '[.migrations[].version]'
> curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"version":2}' \
>   http://127.0.0.1:8282/admin/schema-version | jq .
> sleep 10
[
  "001_canonical_store.sql",
  "002_outbox_unpublished_index.sql"
]
{
  "version": 2
}
```

Gambar 4.1: Mencatat kondisi awal dan memicu schema PVMBG v2 saat sistem berjalan.

Setelah perubahan dipicu, `history-audit -revisions 1,2` dijalankan melalui binary Aggregator untuk membandingkan hasil penyimpanan. Revisi pertama tidak memiliki confidence, sedangkan revisi kedua memiliki nilai 0,58. Keduanya mempertahankan ID hazard dan source reference yang sama. Daftar migrasi tetap dua migrasi awal karena confidence disimpan sebagai tambahan atribut JSONB.

```
$ docker compose exec -T aggregator /aggregator history-audit \
>   --source PVMBG --source-ref-id "$PVMBG_REF" --revisions 1,2 \
>   | tee docs/report/evidence/schema-command-audit.json \
>   | jq '{hazard_id, source_ref_id,
>       versions: [.versions[] | {revision, attributes: .data.attributes}],
>       migrations: [.migrations[].version]}'
>
{
  "hazard_id": "a3e0c470-1b79-4f39-aeb1-c7a063568995",
  "source_ref_id": "sim-19ac08b49fd13778-report-001",
  "versions": [
    {
      "revision": 1,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    },
    {
      "revision": 2,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "confidence_level": 0.58,
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    }
  ],
  "migrations": [
    "001_canonical_store.sql",
    "002_outbox_unpublished_index.sql"
  ]
}
```

Gambar 4.2: Audit schema: identitas hazard tetap, confidence muncul pada revisi kedua, dan daftar migrasi tidak bertambah.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'sort_by(.Name) | map({name: .Name, id: .Id,
>       started_at: .State.StartedAt, restarts: .RestartCount})' >
/tmp/report-session/schema-after.json
> diff -s /tmp/report-session/schema-before.json \
>   /tmp/report-session/schema-after.json
Files /tmp/report-session/schema-before.json and /tmp/report-session/schema-
after.json are identical
```

Gambar 4.3: Perbandingan ID, waktu mulai, dan restart count container sebelum/sesudah toggle menghasilkan file identik pada sesi pengujian [10].

Hasil audit menunjukkan confidence masuk ke revisi baru dengan identitas hazard yang tetap. Perbandingan container juga menunjukkan bahwa perubahan tersebut berlangsung tanpa restart, sementara daftar migrasi membuktikan tidak ada perubahan schema tabel.

### 4.4.2 Pemetaan Bukti ke Kriteria

Hubungan hasil pengujian dengan kriteria P1 dirangkum berikut ini.

| Kriteria | Bukti dan batas interpretasi |
|---|---|
| P1.1 | Pemetaan kedua sumber: tes mapper, contoh command dan trial. |
| P1.2–3 | Pemicu schema v2, audit dua revisi, daftar migrasi, dan kondisi container pada Gambar 4.1–4.3. |
| P1.4 | Field tambahan/nested dan collision: tes mapper; validasi kontrak delapan record pada log pengujian. |
| P1.5 | Mekanisme dan jenis perubahan schema yang didukung maupun tidak didukung pada Bagian 4.5. |

## 4.5. Jawaban atas Butir "Di laporan"

Mekanisme yang dipilih adalah JSON dengan kebijakan **additive-only**. Field wajib tetap divalidasi, sedangkan field tambahan diteruskan ke `attributes`; jika namanya bertabrakan dengan atribut turunan, nilainya ditempatkan pada `source_extras`. Dengan demikian, field yang belum dikenal tidak hilang meskipun belum mempunyai aturan pemetaan tersendiri.

Perubahan yang didukung adalah penambahan field, termasuk nilai nested, selama kontrak field wajib tetap terpenuhi. Penambahan `confidence_level` pada PVMBG v2 menjadi contoh yang diuji. Sistem secara sadar tidak mendukung rename, penghapusan, atau perubahan tipe field wajib tanpa pembaruan kontrak. Field tambahan juga tidak otomatis mengubah severity atau ringkasan client, karena perubahan tersebut memerlukan keputusan domain.

# PROBLEM 2

# CONCURRENCY, AVAILABILITY, & GRACEFUL DEGRADATION

## 5.1. Rancangan Solusi dan Mekanisme

P2 memeriksa apakah sumber yang lambat atau gagal ikut menghambat layanan lain. Rancangan memisahkan pekerjaan polling dari request client, memberi jalur baca kapasitas tersendiri, dan tetap menyajikan data terakhir ketika sumber mengalami outage. Metadata freshness membantu client menilai kondisi data yang diterima.

### 5.1.1 Isolasi request dan polling

Jalur request client dipisahkan dari pengambilan data sumber. Client API membaca data yang telah tersimpan melalui Aggregator, sehingga request tidak harus menunggu mock PVMBG yang dapat terlambat 3 detik. Sementara itu, loop polling BMKG dan PVMBG berjalan sendiri-sendiri; fetch seismic dan warning BMKG juga dapat dilakukan paralel. HTTP client menggunakan kembali koneksi untuk mengurangi biaya pembukaan koneksi pada request berikutnya.

Di database, pekerjaan baca memiliki pool 10 koneksi, sedangkan penulisan BMKG, penulisan PVMBG, dan relay masing-masing mendapat 2 koneksi. Dengan pemisahan ini, pekerjaan polling tidak mengambil seluruh koneksi yang dibutuhkan jalur baca. API dan Aggregator membatasi jumlah request yang sedang diproses menjadi 128. Jika batas tersebut tercapai, service mengembalikan 429 beserta `Retry-After`, sehingga client mengetahui bahwa request perlu dicoba lagi.

Timeout membatasi lamanya setiap pekerjaan: baca database 200 ms, write 2 detik, request API ke Aggregator 1 detik, request ke BMKG 1 detik, dan request ke PVMBG 4 detik. Angka PVMBG dibuat lebih panjang agar delay 3 detik pada pengujian masih dapat ditangani.

### 5.1.2 Degradasi dan pemulihan sumber

Kegagalan sumber tidak selalu berarti data tidak dapat disajikan. Ketika polling PVMBG gagal, Aggregator mempertahankan record terakhir yang berhasil disimpan. API masih mengembalikannya, tetapi `meta.sources.PVMBG.status` menjadi `stale` untuk menunjukkan bahwa pembaruan sumber terganggu. Respons menyertakan `last_success_at` dan `stale_since` sebagai penanda waktu. BMKG dapat tetap `fresh` karena polling-nya berjalan terpisah.

Status `unavailable` digunakan jika sumber belum mempunyai poll yang berhasil penuh maupun record valid yang tersimpan. Jika polling pertama hanya berhasil sebagian, record valid tetap dapat dibaca dengan status stale; `last_success_at` tetap

null sampai satu siklus berhasil penuh. Dengan aturan ini, kegagalan satu record tidak menyembunyikan record lain yang sudah tersimpan.

Untuk pembacaan daftar, status HTTP mengikuti ketersediaan sumber yang diminta. API mengembalikan 503 dengan kode `SOURCE_UNAVAILABLE` jika seluruh sumber yang diminta belum tersedia. Request khusus PVMBG akan memperoleh 503 pada kondisi tersebut, sedangkan request gabungan tetap dapat memperoleh 200 berisi data BMKG jika BMKG tersedia. Metadata tetap menyatakan PVMBG unavailable, sehingga respons gabungan tidak memberi kesan bahwa kedua sumber tersedia.

Setelah outage dinonaktifkan, polling berikutnya memperbarui data dan mengembalikan PVMBG menjadi fresh tanpa restart API atau Aggregator. Selain kegagalan polling, data juga dianggap stale jika waktu sukses terakhir telah melewati ambang 15 detik pada konfigurasi awal.

## 5.2. Alasan Keputusan dan Trade-off

Alternatifnya adalah mengambil data sumber untuk setiap request client. Cara itu dapat memberikan data yang lebih baru, tetapi waktu respons dan kegagalan sumber langsung memengaruhi client. Polling menghindari ketergantungan tersebut dengan konsekuensi adanya jeda pembaruan. Karena itu, API menyertakan metadata freshness agar client mengetahui status sumber dan waktu pembaruan terakhir.

Pool koneksi dan batas request dipilih agar pekerjaan yang lambat tidak menghabiskan seluruh resource. Satu pool bersama lebih sederhana dikonfigurasi, tetapi writer sumber dan request baca akan berebut kapasitas yang sama. Pemisahan pool memberi jalur baca kapasitas tersendiri, dengan konsekuensi total koneksi database harus diperhitungkan. Batas request dan timeout juga dapat menolak atau menghentikan pekerjaan saat beban tinggi, tetapi mencegah antrean tumbuh tanpa batas.

## 5.3. Batasan yang Disadari

Hasil load test menggambarkan beban yang dicoba pada satu host dengan pembacaan maksimal 50 record per request. Pengujian tersebut ditujukan untuk memeriksa ambang beban pada spesifikasi, sehingga belum mencari kapasitas maksimum sistem. Jalur baca tetap membutuhkan Aggregator dan PostgreSQL. Keduanya belum memiliki failover karena deployment M1 memakai satu instance per komponen; isolasi dari gangguan sumber karena itu tidak menjamin API tersedia jika salah satu komponen ini mati.

Penilaian freshness masih menggunakan wall clock. Jika waktu host bergeser, umur data yang ditampilkan dapat ikut berubah meskipun pengukuran latency tetap benar. Perluasan ke beberapa node karena itu memerlukan sinkronisasi waktu serta pengujian ulang terhadap kapasitas dan pola kegagalan yang baru.

## 5.4. Bukti Penyelesaian

Pengujian P2 mencakup outage dan pemulihan sumber, serta load test dengan delay PVMBG 3 detik. Hasilnya ditunjukkan melalui command outage dan berkas pengukuran berikut [10].

Penanganan record valid dari poll parsial juga diperiksa melalui uji regresi `availability_integration_test.go` pada commit `e94e03b`. Uji dengan PostgreSQL ini mencakup BMKG dan PVMBG: data parsial dapat dibaca dengan status stale, sedangkan sumber tanpa record valid tetap menghasilkan 503.

### 5.4.1 Outage dan pemulihan

Pengujian outage dilakukan melalui endpoint kontrol mock. Request berikut mengirim `enabled=true` ke `/admin/outage`; setelah respons diterima, pengujian menunggu polling mendeteksi kegagalan sumber.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":true}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": true
}

$ sleep 10
```

Gambar 5.1: Mengaktifkan outage pada sumber PVMBG.

Berikutnya, dua request dikirim ke Client API dengan token Tim Lapangan. Request pertama memilih PVMBG dan menampilkan jumlah record beserta metadata sumber, sedangkan request kedua membaca metadata BMKG. Hasilnya menunjukkan lima record PVMBG masih dapat dibaca, dengan status stale dan waktu kegagalan yang tercatat. Pada saat yang sama, BMKG tetap fresh. Dengan demikian, outage PVMBG tidak menghilangkan data terakhir atau mengganggu status pembaruan BMKG.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=5' \
>   | jq '{records: (.data | length), source: .meta.sources.PVMBG}'
{
  "records": 5,
  "source": {
    "status": "stale",
    "last_success_at": "2026-10-07T05:15:29.209971Z",
    "stale_since": "2026-10-07T05:15:31.341025Z"
  }
}

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1' \
>   | jq '.meta.sources.BMKG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:36.44186Z",
  "stale_since": null
}
```

Gambar 5.2: Respons API selama PVMBG outage: data terakhir tersedia dan kondisi setiap sumber dinyatakan secara eksplisit.

Pemulihan dilakukan dengan mengirim `enabled=false` ke endpoint yang sama. Setelah jeda polling, request API menunjukkan PVMBG kembali fresh dan `stale_since` menjadi null. Langkah ini tidak menjalankan command restart. Trial integrasi juga membandingkan kondisi container sebelum dan sesudah outage; hasilnya tersimpan di `outage-recovery.json`.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":false}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": false
}

$ sleep 10

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=1' \
>   | jq '.meta.sources.PVMBG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:47.835933Z",
  "stale_since": null
}
```

Gambar 5.3: Mematikan outage dan membaca hasil polling yang sudah pulih.

### 5.4.2 Metode dan hasil load test

Load test mengukur respons API ketika kedua jenis data dibaca bersamaan dan sumber PVMBG diberi delay. Pengujian menggunakan `scripts/load/hazards-monotonic.py`, dengan 30 worker BMKG dan 30 worker PVMBG selama 90 detik. Setiap worker langsung mengirim request berikutnya setelah menerima respons, tanpa think time.

Setiap worker mempunyai sesi Tim Lapangan sendiri, menggunakan HTTP keep-alive, dan membaca `view=full` dengan limit 50. Access token tetap menggunakan TTL 60 detik; script melakukan refresh sebelum expiry agar masa berlaku token tidak menjadi penyebab putusnya beban. Selama pengujian, delay sumber PVMBG ditetapkan 3000 ms.

Latency dihitung sejak request dimulai hingga seluruh body diterima, termasuk pembukaan koneksi jika diperlukan. Pengukuran memakai `time.perf_counter_ns`, yaitu timer monotonic yang tidak mengikuti perubahan wall clock host, sedangkan percentile dihitung dengan metode nearest rank.

Untuk memeriksa syarat koneksi paralel, collector menggunakan Linux `ss` dan mengambil sampel koneksi TCP established ke port API setiap sekitar 1 detik. Pengukuran ini diperlukan karena jumlah worker saja tidak membuktikan jumlah koneksi yang benar-benar terbuka.

Sebelum beban diberikan, delay mock diatur dengan body `min_ms` dan `max_ms` yang sama-sama bernilai 3000. Request langsung ke sumber kemudian mengukur waktu respons melalui `curl`. Flag `-o /dev/null` membuang body, sedangkan `-w` menampilkan status HTTP dan durasi request. Pengukuran ini membedakan delay sumber dari latency Client API yang membaca data tersimpan.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' \
>   --data '{"min_ms":3000,"max_ms":3000}' \
>   http://127.0.0.1:8282/admin/delay | jq .
> curl -sS -o /dev/null -w 'HTTP %{http_code}; durasi %{time_total} s\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
{
  "min_ms": 3000,
  "max_ms": 3000
}
HTTP 200; durasi 3.018729 s
```

Gambar 5.4: Mengatur delay 3 detik dan mengukur request langsung ke sumber PVMBG.

Collector TCP dijalankan bersamaan dengan load test untuk mengukur jumlah koneksi established selama beban berlangsung. Kedua proses dimulai dari root repo dengan command berikut:

```
export API_URL="http://127.0.0.1:${API_HTTP_PORT:-8080}"
export AUTH_URL="http://127.0.0.1:${AUTH_HTTP_PORT:-8084}"
python3 scripts/load/capture-tcp-connections.py \
  --api-url "$API_URL" --seconds 110 --interval 1 \
  --output /tmp/report-tcp.json &
tcp_pid=$!
python3 scripts/load/hazards-monotonic.py \
  --seconds 90 --vus-per-source 30 --output /tmp/report-load.json
load_exit=$?
wait "$tcp_pid"
tcp_exit=$?
test "$load_exit" -eq 0 && test "$tcp_exit" -eq 0
```

Hasil load test tersimpan di `docs/report/evidence/load.json`. Field yang diperiksa meliputi jumlah worker, durasi pengujian, p95 BMKG, throughput, dan persentase error.

```
$ jq '{started_at, profile, elapsed_seconds,
>     bmkg_p95_ms: .sources.BMKG.success_duration_ms.p95,
>     throughput: .hazard_successes_per_second,
>     error_percent: .hazard_unexpected_error_percent}' \
>   docs/report/evidence/load.json
{
  "started_at": "2026-10-05T13:11:41.596066+00:00",
  "profile": {
    "vus_per_source": 30,
    "duration_seconds": 90.0,
    "view": "full",
    "limit": 50,
    "identity": "field-team",
    "access_ttl_seconds": 60
  },
  "elapsed_seconds": 90.08667208399856,
  "bmkg_p95_ms": 124.521589,
  "throughput": 980.25599078252,
  "error_percent": 0.03396047001290498
}
```

Gambar 5.5: Membaca hasil load test dari berkas pengukuran yang disimpan.

| Metrik | BMKG | PVMBG |
|---|---|---|
| Request terkirim | 44.510 | 43.828 |
| Respons sukses | 44.494 | 43.814 |
| Error selain 429 | 16 | 14 |
| HTTP 429 pada API | 0 | 0 |
| p50 sukses (ms) | 52,07 | 52,64 |
| p95 sukses (ms) | 124,52 | 125,24 |
| p99 sukses (ms) | 190,71 | 194,30 |
| p95 seluruh respons (ms) | 124,76 | 125,44 |

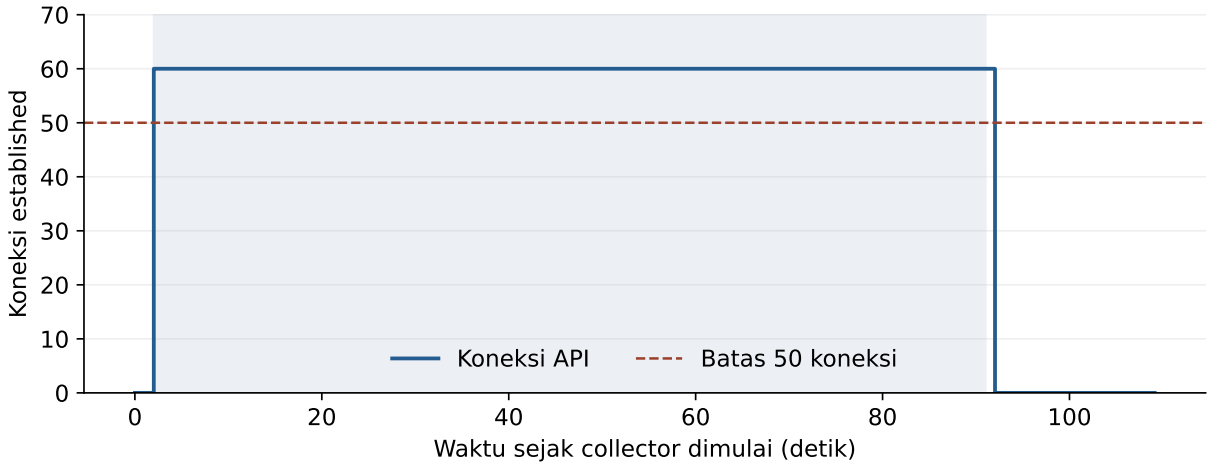
Tabel 5.1: Hasil beban simultan selama 90,09 detik. p50/p95/p99 sukses dihitung dari 2xx; p95 seluruh respons juga ditampilkan.

Dari 88.338 request yang dikirim, 88.308 mendapat respons sukses. Throughput-nya mencapai 980,26 respons sukses per detik, sedangkan 30 request lainnya menghasilkan HTTP 503, sebagaimana dicatat dalam `docs/report/evidence/load-statuses.json`. Dengan demikian, error selain 429 sebesar 0,034%, masih di bawah batas 1%.

API tidak menghasilkan 429, sedangkan Auth menghasilkan 18 respons 429 selama

login dan refresh. Retry berhasil sehingga seluruh sesi worker tetap berjalan. Respons Auth dicatat terpisah dari error pembacaan hazard.

Delay PVMBG terukur 3.004,46 ms, sedangkan p95 BMKG 124,52 ms memenuhi batas 300 ms saat kedua sumber dibaca bersamaan. Collector mencatat sedikitnya 50 koneksi pada rangkaian sampel selama 89,00 detik, melebihi syarat 60 detik. Pengambilan sampel sekitar sekali per detik menentukan resolusi bukti koneksi tersebut.




Gambar 5.6: Sampel koneksi established selama pengujian. Garis putus-putus menunjukkan batas 50 koneksi. Interval di atas batas dihitung dari rangkaian sampel koneksi.

| Kriteria | Bukti |
|---|---|
| P2.1 | Delay sumber terukur dan BMKG p95 pada Tabel 5.1; kedua sumber menerima request bersamaan. |
| P2.2 | Throughput, percentile, error, 429, serta sampel TCP; Tabel 5.1 dan Gambar 5.6. |
| P2.3 | Kontrol outage, data stale, dan pemulihan pada Gambar 5.1–5.3; trial lengkap di `outage-recovery.json`. |
| P2.4 | Model konkurensi dan penanganan tiga kegagalan pada Bagian 5.5; metode, command, dan hasil load test pada bagian bukti. |

## 5.5. Jawaban atas Butir "Di laporan"

Model konkurensi menggunakan handler HTTP Go untuk request client dan loop polling terpisah untuk BMKG serta PVMBG. Tiga kegagalan pada P2 ditangani sebagai berikut:

1. **Head-of-line blocking**: request membaca Canonical Store, bukan menunggu fetch sumber. Polling PVMBG yang lambat juga tidak menahan loop BMKG.
2. **Resource exhaustion**: pool baca dipisahkan dari writer dan relay, jumlah request aktif dibatasi 128, dan setiap operasi memiliki timeout. Request di luar kapasitas ditolak secara terkendali dengan 429.
3. **Ketiadaan degradasi terkendali**: data tersimpan tetap disajikan dengan penanda stale. Sumber tanpa data dinyatakan unavailable, dan polling memulihkan freshness setelah sumber pulih.

Load test menggunakan 30 worker per sumber selama 90 detik, HTTP keep-alive, limit 50, dan delay PVMBG 3 detik. Latency diukur dengan timer monotonic dan koneksi TCP diperiksa melalui `ss`. Tool serta command tersedia pada Bagian 5.4.2; hasilnya pada Tabel 5.1 dan Gambar 5.6.

# PROBLEM 3

# SERVICE-TO-SERVICE AUTHENTICATION & TRUST

## 6.1. Rancangan Solusi dan Mekanisme

Pada Problem 3, akses ke sumber data, service internal, dan API client perlu dibatasi sesuai identitas serta hak masing-masing. Credential yang sah untuk satu sumber tidak boleh membuka sumber lain, dan identitas Media tidak boleh memperoleh data raw. Rancangan menggunakan credential per domain, pembatasan scope di server, serta access token berumur pendek yang dapat diperbarui tanpa login ulang.

### 6.1.1 Credential dan hak akses

Setiap antarmuka memiliki credential sesuai perannya. BMKG menerima API key melalui header `X-BMKG-Key`, PVMBG menerima Bearer token sumber, dan API internal Aggregator menggunakan key lain. Dengan pemisahan ini, credential satu instansi tidak dapat digunakan untuk mengakses instansi lain atau sebagai identitas client.

Di sisi client, Auth membedakan identitas `media`, `field-team`, dan `bnpb-internal`, masing-masing dengan secret sendiri. Media hanya mendapat scope ringkasan, yang berisi tepat tujuh field: `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Tim Lapangan dan BNPB Internal mendapat akses lengkap, termasuk `source_ref_id`, koordinat, serta `attributes`.

Client API memeriksa scope sebelum meminta data ke Aggregator dan menyaring field pada respons. Karena aturan berada di server, Media tidak dapat melewatinya dengan mengubah query menjadi `view=full` atau meminta atribut raw. Kedua bentuk request tersebut ditolak dengan 403.

### 6.1.2 Penerbitan dan verifikasi token

Setelah identitas client diperiksa, Auth menerbitkan JWT [11] yang ditandatangani dengan algoritma RS256 menggunakan private key. Client API hanya memegang public key untuk memverifikasi signature, issuer, audience, expiry, dan scope. Pemisahan key ini memungkinkan API memeriksa token tanpa memiliki kemampuan untuk menerbitkannya.

Verifikasi dilakukan secara lokal pada setiap request data. Karena itu, token yang masih valid tetap dapat dipakai ketika Auth berhenti sementara. Auth baru diperlukan kembali untuk login atau refresh; perilaku ini juga diperiksa pada trial integrasi [10].

### 6.1.3 Expiry dan refresh

Access token memiliki masa berlaku default 60 detik, yang dapat diatur melalui `ACCESS_TOKEN_TTL`. TTL singkat membatasi masa penggunaan token yang bocor,

tetapi client perlu cara melanjutkan sesi setelah token berakhir. Untuk itu, login juga menghasilkan refresh token berupa nilai acak yang dapat ditukar melalui Auth dengan pasangan token baru.

Hanya hash refresh token yang disimpan di SQLite. Pada setiap refresh, token lama ditandai sudah digunakan dan diganti dengan token baru. Jika token lama dipakai kembali, keluarga sesi dibatalkan untuk menghentikan kelanjutan refresh melalui token tersebut. Database Auth berada pada volume privat agar state sesi bertahan ketika container dijalankan ulang.

Keluarga sesi refresh memiliki batas umur default 24 jam, yang dapat diatur melalui `REFRESH_TOKEN_TTL`. Batas ini dihitung sejak sesi dibentuk dan tidak diperpanjang oleh rotasi. Kedua TTL harus positif; profil pengujian menggunakan access token 60 detik dan sesi refresh 24 jam.

## 6.2. Alasan Keputusan dan Trade-off

Pilihan verifikasi token mempertimbangkan ketergantungan jalur baca dan kepemilikan key. Jika Auth dan API berbagi signing secret, material verifikasi pada API juga dapat dipakai untuk membuat token. RS256 memisahkan kedua kemampuan tersebut. Alternatif introspection ke Auth pada setiap request memudahkan pemeriksaan status token secara terpusat, tetapi membuat pembacaan data ikut bergantung pada ketersediaan Auth. Verifikasi lokal dipilih untuk menjaga jalur baca, dengan konsekuensi pencabutan access token tidak langsung diketahui API.

Refresh token memungkinkan access token tetap berumur pendek tanpa meminta client login ulang setiap 60 detik. Memperpanjang TTL access token lebih sederhana, tetapi juga memperpanjang kesempatan menggunakan token yang bocor. Rotasi dan penyimpanan hash refresh token dipilih untuk membatasi penggunaan ulang serta risiko kebocoran database. Trade-off-nya adalah kebutuhan state sesi persisten dan ketersediaan Auth saat refresh.

## 6.3. Batasan yang Disadari

Verifikasi lokal belum dilengkapi daftar revocation access token. Karena itu, rotasi atau pembatalan sesi refresh tidak langsung mencabut JWT yang sudah diterbitkan; token tersebut tetap dapat digunakan sampai `exp`. TTL 60 detik membatasi interval ini, sedangkan pencabutan segera memerlukan mekanisme revocation atau pemeriksaan ke Auth.

Expiry bergantung pada wall clock host. Jika clock mundur, token dapat terasa berlaku lebih lama; jika clock maju, token dapat berakhir lebih cepat. API memeriksa signature, issuer, audience, scope, expiry, serta `nbf` jika ada, tetapi tidak menolak token hanya karena `iat` tampak berada di masa depan setelah clock mundur. Pada profil pengujian, selisih `exp-iat` tetap 60 detik. Agar interval tersebut sesuai waktu nyata, sinkronisasi waktu host perlu dijaga.

Perlindungan koneksi masih dibatasi pada demo lokal. HTTP internal belum menggunakan TLS, NATS menggunakan token bersama, dan rotasi key RSA belum diterapkan. Jaringan Docker serta binding port ke loopback membatasi paparan, tetapi belum cukup untuk deployment pada jaringan yang tidak dipercaya. Deployment terse-

but membutuhkan TLS, credential broker per service, dan pengelolaan rotasi key.

## 6.4. Bukti Penyelesaian

Pengujian P3 mengikuti tiga batas akses: credential sumber, field yang boleh dibaca client, dan masa berlaku token. Buktinya ditunjukkan melalui command berikut dan hasil trial integrasi [10].

### 6.4.1 Credential dan hak akses

Pemeriksaan dimulai dari akses ke mock, lalu dilanjutkan dengan akses Media melalui API. Urutan ini memperlihatkan pemisahan autentikasi sumber dan otorisasi client.

#### Credential hanya berlaku pada sumbernya

Pengujian credential dilakukan sebagai empat request terpisah. Untuk BMKG, header tetap bernama `X-BMKG-Key`: request pertama mengisinya dengan key BMKG, sedangkan request kedua memakai token PVMBG sebagai nilai yang salah. Status HTTP setiap request dibaca melalui opsi `-w`.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $BMKG_API_KEY" \
>   http://127.0.0.1:8281/seismic-events
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $PVMBG_TOKEN" \
>   http://127.0.0.1:8281/seismic-events
HTTP 401
```

Gambar 6.1: BMKG menerima key miliknya dengan 200 dan menolak credential PVMBG dengan 401.

Arah sebaliknya diuji pada PVMBG dengan schema Bearer yang sama untuk kedua request. Token PVMBG menghasilkan 200, sedangkan key BMKG menghasilkan 401. Karena format header tetap sesuai dengan antarmuka sumber, status 401 menunjukkan bahwa sumber menolak nilai credential yang salah.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $BMKG_API_KEY" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 401
```

Gambar 6.2: PVMBG menerima token miliknya dan menolak key BMKG.

**Media menerima ringkasan dan tidak dapat meminta data raw**

Media terlebih dahulu login ke Auth dengan identitas dan secret client-nya sendiri. Respons login disimpan pada file privat untuk digunakan pada request berikutnya. Scope `hazards:summary` menjadi dasar pembatasan di API.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data
"{\"client_id\":\"media\",\"client_secret\":\"$MEDIA_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-
session/media.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/media.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary"
}
```

Gambar 6.3: Auth memberikan scope ringkasan dan TTL 60 detik untuk Media.

Access token dari respons login dipasang pada header Authorization ketika Media membaca satu record BMKG. Filter `keys` menampilkan daftar field yang diterima: jumlahnya tepat tujuh dan tidak mencakup koordinat, `source_ref_id`, maupun `attributes`.

```
$ MEDIA_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/media.json)
> curl -sS -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&limit=1' \
>   | jq '.data[0] | keys'
[
  "area_name",
  "hazard_id",
  "hazard_type",
  "ingested_at",
  "occurred_at",
  "severity",
  "source"
]
```

Gambar 6.4: Field respons Media sesuai dengan kontrak ringkasan.

Media kemudian mencoba `view=full` dan `fields=attributes` melalui dua request tersendiri. Keduanya ditolak dengan 403. Hasil ini menunjukkan bahwa query yang diubah client tidak dapat melewati pembatasan server.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?view=full'
HTTP 403

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?fields=attributes'
HTTP 403
```

Gambar 6.5: Dua cara meminta data raw sama-sama ditolak untuk token Media.

### 6.4.2 Expiry dan refresh

Alur berikut menggunakan satu sesi Tim Lapangan untuk membandingkan akses sebelum expiry, penolakan setelah expiry, dan pemulihan akses melalui refresh.

Login Tim Lapangan menghasilkan scope `hazards:summary hazards:raw` dan access token berumur 60 detik. Access token dari respons login digunakan pada request API berikutnya.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data "{\"client_id\":\"field-
team\",\"client_secret\":\"$FIELD_TEAM_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-session/field-
team.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/field-
team.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.6: Login Tim Lapangan memberikan izin ringkasan dan data raw.

Command berikut membaca payload JWT untuk menampilkan `iat` dan `exp`, kemudian menghitung selisihnya. Decoding pada terminal hanya digunakan untuk inspeksi claim; verifikasi token tetap dilakukan API ketika request dikirim. Selisihnya 60 detik dan token yang belum expired memperoleh 200.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | {iat, exp, ttl: (.exp-
.iat)}'
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
{
  "iat": 1791349941,
  "exp": 1791350001,
  "ttl": 60
}
HTTP 200
```

Gambar 6.7: Memeriksa TTL dari claim dan menggunakan access token sebelum expiry.

Selanjutnya, loop menunggu wall clock melewati `exp` dari token yang sama. Request API setelah batas tersebut menghasilkan 401.

```
$ ACCESS_EXP=$(jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | .exp')
> while [ "$(date +%s)" -le "$ACCESS_EXP" ]; do sleep 1; done
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.8: Token lama ditolak setelah waktu expiry terlewati.

Refresh dilakukan dengan mengambil `refresh_token` dari file login dan mengirimkannya sebagai body JSON ke Auth. Opsi `-data-binary @-` mengirim body JSON dari pipeline, yang berisi refresh token dari respons login. Respons baru disimpan pada file lain dan menunjukkan TTL 60 detik serta scope yang sama. Tidak ada request login pada langkah ini.

```
$ jq '{refresh_token}' .secrets/report-session/field-team.json \
>   | curl -sS --fail-with-body -H 'Content-Type: application/json' \
>       --data-binary @- http://127.0.0.1:8284/v1/auth/refresh \
>       -o .secrets/report-session/refreshed.json
> jq '{token_type, expires_in, scope}' .secrets/report-
session/refreshed.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.9: Mendapatkan token baru melalui refresh tanpa login ulang.

Token baru kemudian mendapat 200 ketika digunakan ke API, sementara token lama tetap mendapat 401. Trial integrasi melengkapi alur ini dengan pemeriksaan sesi refresh setelah restart Auth, replay refresh token, dan penggunaan JWT yang masih valid ketika Auth berhenti. Hasilnya tersedia pada `integration.json`.

```
$ NEW_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/refreshed.json)
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.10: Token baru dapat digunakan, sedangkan access token lama tetap expired.

Tabel berikut menghubungkan hasil pengujian dan pemeriksaan konfigurasi dengan kriteria P3.

| Kriteria | Bukti |
|---|---|
| P3.1 | Empat request pada Gambar 6.1 dan 6.2. |
| P3.2 | Tujuh field Media dan penolakan 403 pada Gambar 6.4 serta 6.5. |
| P3.3 | TTL, expiry alami, refresh, serta pemeriksaan token lama dan baru pada Gambar 6.7–6.10. |
| P3.4 | Secret pada `.env.example` kosong; `.env` dan `.secrets/` diabaikan Git serta dikecualikan dari build context. Key dan sesi tersimpan pada volume privat. |
| P3.5 | Mekanisme pemisahan credential, verifikasi JWT, dan perlindungan refresh token pada Bagian 6.5, beserta batas keamanannya. |

### 6.4.3 Menelusuri request dengan correlation ID

Selain hasil respons, alur request dapat ditelusuri melalui log ber-correlation ID. Pada contoh berikut, client mengirim header `X-Correlation-ID`, lalu Client API meneruskannya ke Aggregator. ID yang sama menghubungkan catatan dari kedua service, sehingga operasi di balik satu request dapat diperiksa.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   -H 'X-Correlation-ID: report-command-correlation' \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
>
HTTP 200
```

Gambar 6.11: Request client dengan correlation ID yang ditentukan secara eksplisit.

Command log kemudian memilih catatan dengan ID tersebut. Keluaran menunjukkan operasi API, pembacaan upstream, dan operasi Aggregator yang semuanya memiliki correlation ID sama.

```
$ docker compose logs --no-color --no-log-prefix --since 2m \
>   client-api aggregator \
>   | jq -R 'fromjson? | select(.correlation_id == "report-command-
correlation")
>       | {service, correlation_id, operation, status, duration_ms}'
>
{
  "service": "aggregator",
  "correlation_id": "report-command-correlation",
  "operation": "GET /internal/v1/hazards",
  "status": 200,
  "duration_ms": 2
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "aggregator_read",
  "status": 200,
  "duration_ms": 3
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "GET /v1/hazards",
  "status": 200,
  "duration_ms": 4
}
```

Gambar 6.12: Log kedua service dapat ditelusuri melalui correlation ID yang sama.

## 6.5. Jawaban atas Butir “Di laporan”

Pemisahan domain of trust dilakukan dengan credential berbeda untuk BMKG, PVMBG, API internal Aggregator, dan ketiga identitas client. Client API memeriksa scope di server sebelum mengambil data, sehingga credential Media tidak dapat digunakan untuk memperoleh field raw.

JWT RS256 memungkinkan API memverifikasi token dengan public key tanpa memperoleh private key penerbitnya. API memeriksa signature, issuer, audience, expiry, dan scope, serta membatasi algoritma verifikasi. Refresh token disimpan sebagai hash, dirotasi setiap penggunaan, dan memiliki batas umur sesi; penggunaan ulang token refresh membatalkan keluarga sesi tersebut. Credential dan key disediakan melalui konfigurasi privat, bukan di-hardcode pada source atau image.

Perlindungan ini masih dibatasi oleh access JWT yang belum dapat dicabut langsung, serta TLS internal dan rotasi key yang belum diterapkan. Konsekuensinya dibahas pada bagian Batasan yang Disadari.

# PROBLEM 4

# ARSITEKTUR MICROSERVICE & FLEXIBLE STORAGE

## 7.1. Rancangan Solusi dan Mekanisme

P4 menguji dua bentuk kemandirian sistem: satu service dapat diganti tanpa restart seluruh stack, dan data dapat menerima atribut baru tanpa perubahan schema tabel. Rancangan memisahkan container berdasarkan tanggung jawab, menetapkan satu pemilik untuk setiap storage, dan memakai JSONB untuk atribut sumber yang dinamis.

### 7.1.1 Container dan kepemilikan storage

Stack utama terdiri dari tujuh container aplikasi serta dua container infrastruktur, yaitu PostgreSQL dan NATS. Setiap aplikasi memiliki Dockerfile sendiri, sedangkan infrastruktur memakai image resmi. Compose mengatur jaringan dan startup bersama, sekaligus menyediakan operasi per service. Dengan `-no-deps`, satu aplikasi dapat dijalankan kembali tanpa ikut menjalankan ulang dependency-nya. Third Consumer tersedia melalui profil demo untuk menguji penambahan pembaca event.

Canonical Store dikelola oleh Aggregator. Batas ini diterapkan pada jaringan: hanya Aggregator dan PostgreSQL yang terhubung ke `store-net`, dan PostgreSQL tidak memublikasikan port ke host. Client API membaca melalui HTTP Aggregator, sedangkan consumer memperoleh data melalui NATS. Hasil olahan setiap consumer disimpan pada database privatnya sendiri. Untuk audit riwayat canonical, perintah dijalankan melalui binary Aggregator sebagai pemilik store.

### 7.1.2 Canonical Store

PostgreSQL menyimpan field kanonik dengan tipe dan constraint, serta atribut khusus sumber dalam JSONB [5]. Penyimpanan dibagi berdasarkan fungsi pada Tabel 7.1, agar data terbaru, riwayat, status sumber, dan kebutuhan publish tetap dapat dikelola bersama.

| Tabel | Fungsi |
|---|---|
| `hazard_events` | State terbaru setiap hazard, dengan unique key pasangan sumber dan reference ID. |
| `hazard_versions` | Snapshot append-only untuk setiap revisi hazard. |
| `source_state` | Status percobaan polling, sukses, kegagalan, dan freshness sumber. |
| `outbox` | Envelope event immutable beserta status pengirimannya ke broker. |

Tabel 7.1: Pembagian data dalam Canonical Store milik Aggregator.

Ketika hazard berubah, pembaruan data terbaru, penambahan versi, dan pencatatan outbox dilakukan dalam satu transaksi. Dengan demikian, revisi yang tersimpan juga memiliki catatan event untuk dikirim. Jika isinya identik secara semantik, termasuk angka JSON yang ekuivalen, revisi tidak ditambah. Kapasitas pending outbox diperiksa dalam transaksi yang sama; snapshot yang melampaui batas tidak di-commit.

Migrasi schema dijalankan Aggregator saat startup dengan version, advisory lock, dan checksum. Penambahan `confidence_level` pada PVMBG v2 hanya mengubah isi atribut JSONB, sehingga tidak memerlukan migrasi baru. Riwayat sebelum dan sesudah perubahan dapat dibaca melalui CLI `history-audit` pada binary Aggregator. Jalur ini menyediakan inspeksi melalui service pemilik data, sementara API publik tetap berfokus pada data terbaru.

## 7.2. Alasan Keputusan dan Trade-off

Pemisahan service dipilih untuk menjaga independensi operasi dan akses data. Pilihan storage melengkapi batas tersebut dengan menyediakan fleksibilitas atribut serta transaksi dalam satu Canonical Store. Kedua keputusan ini dibandingkan dengan alternatifnya berikut.

### 7.2.1 Batas service

Batas service mengikuti bagian yang perlu berubah atau mengalami gangguan secara independen. Mock BMKG dan PVMBG dipisahkan agar delay, outage, dan credential setiap instansi dapat diuji sendiri. Menggabungkan keduanya dalam satu proses lebih sederhana, tetapi mengurangi isolasi kegagalan sumber.

Polling, mapping, query, dan relay tetap berada pada Aggregator karena mengelola Canonical Store yang sama. Memecah ingest per sumber memberi isolasi deployment tambahan, tetapi membutuhkan antarmuka dan koordinasi storage baru. Pada M1, loop serta pool koneksi dipisahkan di dalam satu service pemilik data.

Client API dipisahkan agar perubahan penyajian dan hak akses tidak mengharuskan rebuild jalur ingest. Auth menjadi service tersendiri untuk menjaga private key penerbit token; API cukup memiliki public key. Dibanding satu unit deploy, pembagian ini menambah konfigurasi dan dependency jaringan, tetapi membatasi tanggung jawab serta material rahasia setiap service.

Dashboard dan Notifier juga dipisahkan karena hasil olahannya berbeda. Container, durable, dan database privat memungkinkan satu consumer diganti ketika consumer lain tetap berjalan. Uraian alur antarservice terdapat pada Bab 2.

Alternatif shared database dapat mengurangi kebutuhan antarmuka, tetapi membuat beberapa service bergantung pada schema tabel yang sama dan bertentangan dengan ownership yang diminta spesifikasi. API dan kontrak event dipilih untuk menjaga batas tersebut. Konsekuensinya, state consumer diperbarui secara eventual: perubahan data kanonik harus melewati outbox dan broker sebelum diterapkan di consumer.

### 7.2.2 Alternatif penyimpanan

Pilihan storage perlu memenuhi fleksibilitas atribut sekaligus konsistensi revisi dan event. PostgreSQL dengan JSONB mempertahankan constraint dan transaksi, sambil menerima tambahan field pada dokumen atribut. Tabel 7.2 membandingkannya dengan kolom relasional tetap dan document store.

| Pilihan | Kelebihan | Konsekuensi |
|---|---|---|
| PostgreSQL + JSONB (dipilih) | Constraint identitas, query canonical, dan transaksi outbox dalam satu store. | Atribut dinamis tetap perlu validasi; query JSONB yang sering dipakai dapat membutuhkan index tambahan. |
| Relasional dengan kolom tetap | Tipe setiap atribut jelas dan mudah di-query. | Field baru membutuhkan migrasi, sehingga kurang sesuai dengan perubahan schema langsung pada P4. |
| Document store, misalnya MongoDB | Bentuk dokumen sumber mudah disimpan dan berubah. | Aturan canonical, index, dan transaksi history/outbox tetap perlu dirancang. |

Tabel 7.2: Alternatif storage untuk atribut dinamis dan transaksi data.

## 7.3. Batasan yang Disadari

Isolasi diterapkan pada akses aplikasi dan operasi container. Administrator Docker tetap dapat mengakses container serta volume, sehingga administrator host berada di luar batas kontrol aplikasi. Aggregator dan PostgreSQL juga masih menggunakan satu instance; penggantian satu consumer tidak menjamin API tersedia jika salah satu pemilik jalur baca tersebut mati.

History, inbox, dan outbox terpublikasi dipertahankan untuk audit serta deduplikasi selama pengujian. Pembersihan jangka panjang belum diterapkan, sehingga data lokal dapat terus bertambah. Kebijakan retention lokal dan penanganan snapshot besar pada P1 perlu disiapkan sebelum volume sistem ditingkatkan.

## 7.4. Bukti Penyelesaian

Pengujian P4 mengganti Dashboard sambil memantau API dan kondisi container lain, lalu memeriksa batas akses ke storage. Fleksibilitas JSONB diperiksa bersama perubahan schema pada P1. Buktinya ditunjukkan melalui command berikut dan hasil trial integrasi [10].

### 7.4.1 Penggantian satu container

Untuk membuktikan bahwa service lain tidak ikut dijalankan ulang, pengujian terlebih dahulu mencatat ID, waktu mulai, dan restart count seluruh container selain Dashboard. Catatan ini akan dibandingkan setelah Dashboard diganti.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
before.json
```

Gambar 7.1: Mencatat kondisi container lain sebelum Dashboard dibangun ulang.

Dashboard kemudian dihentikan dan image-nya dibangun kembali. Log build disimpan ke file, lalu `tail -n 6` membaca bagian akhir yang menunjukkan image berhasil dibangun. Log lengkap tersedia pada `evidence/dashboard-rebuild.log`.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ docker compose build dashboard-consumer \
>   > .secrets/report-session/rebuild.log 2>&1
> tail -n 6 .secrets/report-session/rebuild.log

#15 [dashboard-consumer] exporting to image
#15 exporting layers done
#15 writing image
sha256:c54f9851d0aa3b832644178aac40303d6fd0d5e90ffaaecaa5989e5071680245 done
#15 naming to docker.io/library/aat-report-revision-dashboard-consumer done
#15 DONE 0.0s
```

Gambar 7.2: Menghentikan Dashboard dan membangun image service tersebut.

Saat Dashboard masih berhenti, request BMKG ke API menghasilkan 200. Dashboard kemudian dijalankan kembali dengan `-no-deps -force-recreate`; opsi pertama menjaga dependency tidak ikut dijalankan ulang, sedangkan opsi kedua memastikan container Dashboard diganti. `-wait` menunggu hasil healthcheck-nya siap.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ docker compose up -d --no-deps --force-recreate --wait dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Recreate
 Container aat-report-revision-dashboard-consumer-1  Recreated
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started
 Container aat-report-revision-dashboard-consumer-1  Waiting
 Container aat-report-revision-dashboard-consumer-1  Healthy
```

Gambar 7.3: API tetap merespons saat Dashboard mati, lalu Dashboard diganti secara terpisah.

Inspeksi diulang dengan proyeksi yang sama, kemudian `diff -s` membandingkan file sebelum dan sesudah. Keduanya identik dan berisi delapan container lain. Jadi, penggantian Dashboard tidak mengubah ID, waktu mulai, maupun restart count service lainnya. Trial integrasi juga mencatat 10 request API sukses selama rebuild; rincian pengujian tersebut tetap tersedia di `integration.json`.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
after.json
> diff -s .secrets/report-session/rebuild-before.json \
>   .secrets/report-session/rebuild-after.json
> jq length .secrets/report-session/rebuild-after.json
Files .secrets/report-session/rebuild-before.json and .secrets/report-
session/rebuild-after.json are identical
8
```

Gambar 7.4: Delapan container lain tidak berubah setelah Dashboard diganti.

### 7.4.2 Ownership dan flexible storage

Batas kepemilikan Canonical Store diperiksa melalui konfigurasi jaringan dan jalur akses pada kode. Inspeksi menunjukkan bahwa `store-net` hanya memuat Aggregator dan PostgreSQL, serta port database tidak dipublikasikan ke host. Kode Client API membaca melalui HTTP Aggregator, sedangkan consumer membaca dari NATS; keduanya tidak membuka koneksi SQL ke Canonical Store.

```
$ docker network inspect "${COMPOSE_PROJECT_NAME}_store-net" \
>   | jq '.[0].Containers | [.[] | .Name] | sort'
> docker inspect $(docker compose ps -q postgres) \
>   --format '{{json .NetworkSettings.Ports}}'
>
[
  "aat-report-revision-aggregator-1",
  "aat-report-revision-postgres-1"
]
{}
```

Gambar 7.5: Anggota jaringan storage dan port PostgreSQL yang tidak dipublikasikan.

Fleksibilitas storage dibuktikan bersama pengujian P1 pada Gambar 4.2. Snapshot revisi 1 dan 2 tetap terbaca dengan daftar migrasi yang sama. Pembacaan tersebut dilakukan melalui `/aggregator history-audit`, sehingga perubahan dokumen diperiksa oleh service pemilik storage.

Kriteria P4 dan bukti yang mendukungnya dirangkum pada tabel berikut.

| Kriteria | Bukti |
|---|---|
| P4.1 | Rebuild, request API, dan perbandingan delapan container pada Gambar 7.1–7.4. |
| P4.2 | Revisi v1/v2 dan daftar migrasi pada Gambar 4.2. |
| P4.3 | Anggota `store-net`, port privat, serta jalur baca pada kode; Gambar 7.5. |
| P4.4 | Alasan batas service pada Bagian 7.5 dan perbandingan tiga pilihan storage pada Tabel 7.2. |

### 7.4.3 Verifikasi komponen storage

Tes storage pada commit `19beead` lulus untuk UUID stabil, revisi/history, idempotensi, kapasitas outbox, presisi angka, serta round-trip JSONB ke JetStream. Lingkungan native pengujiannya dicatat pada Bab 3.

## 7.5. Jawaban atas Butir “Di laporan”

Batas service mengikuti pemilik storage dan kebutuhan deploy independen. Dengan pembagian pada Bab 2, perubahan akses atau consumer tidak memerlukan deploy ulang ingest. Ingest per sumber belum dipisah karena menambah koordinasi storage, sedangkan shared database bertentangan dengan ownership yang disyaratkan.

Dari tiga alternatif pada Tabel 7.2, PostgreSQL dengan JSONB dipilih untuk menerima atribut baru tanpa migrasi sekaligus menyediakan constraint dan transaksi data, history, serta outbox. Trade-off-nya adalah validasi atribut dinamis dan kemungkinan kebutuhan index tambahan.

# PROBLEM 5

# FAN-OUT EVENT DISTRIBUTION/PUB-SUB

## 8.1. Rancangan Solusi dan Mekanisme

Pada Problem 5, event dari satu producer perlu diterima oleh beberapa consumer, termasuk consumer yang sempat berhenti atau baru ditambahkan. Rancangan menggunakan broker persisten sebagai perantara. Setiap consumer memiliki posisi baca sendiri, sehingga dapat mengejar event setelah pulih dan tidak mengambil jatah pesan consumer lain.

### 8.1.1 Producer, stream, dan consumer

Alur distribusi dimulai ketika Aggregator menyimpan perubahan hazard. Data kanonik, revisi, dan pesan outbox ditulis dalam satu transaksi, sehingga perubahan yang tersimpan mempunyai catatan untuk dikirim. Relay kemudian menerbitkan envelope ke subject `hazard.events.v1.BMKG` atau `hazard.events.v1.PVMBG`. Stream `HAZARD_EVENTS` menyimpan pesan dari kedua subject melalui filter `hazard.events.v1.*`.

Producer mempublikasikan pesan berdasarkan subject dan kontrak event, sehingga penambahan pembaca tidak memerlukan perubahan pada producer. Daftar consumer serta posisi baca masing-masing dikelola oleh broker.

Dashboard dan Notifier memakai durable yang berbeda, yaitu `dashboard-v1` dan `notifier-v1`. Broker menyimpan posisi baca setiap durable meskipun aplikasinya berhenti. Dengan posisi yang terpisah, masing-masing aplikasi dapat menerima seluruh event dan memberikan ACK sendiri.

Konfigurasi berikut mengatur berapa lama event tersedia, kapan pesan dikirim ulang, dan berapa banyak pesan yang dapat diproses bersamaan. Batas retention menentukan kesempatan consumer yang pulih atau baru ditambahkan untuk membaca event lama.

| Konfigurasi | Nilai dan fungsi |
|---|---|
| Storage / replica / retention | File / 1 / LimitsPolicy; ACK consumer tidak menghapus event dari stream. |
| Batas retention | 24 jam, 256 MiB, 100.000 pesan; batas yang tercapai lebih dahulu menentukan kapasitas. |
| Batas satu pesan | 64 KiB, termasuk ukuran yang diperhitungkan publisher. |
| Saat kapasitas penuh | DiscardNew; publish baru ditolak, bukan menghapus pesan lama hanya untuk menerima pesan baru. |
| Dedup broker | Message ID `hazard_id:revision`; window 2 menit. |
| ACK / redelivery | AckExplicit, AckWait 30 detik, MaxDeliver $-1$; retry tidak dibatasi jumlahnya. |
| Pull / inflight | Batch 10, expiry fetch 2 detik, MaxAckPending 64. |

Tabel 8.1: Konfigurasi JetStream dan consumer yang digunakan pada M1.

### 8.1.2 Semantik pengiriman dan idempotensi

Semantik pengiriman sistem adalah **at-least-once**, sehingga sebuah pesan dapat diterima lebih dari sekali. Misalnya, broker sudah menerima publish, tetapi relay gagal mencatat status published di database. Relay akan mencoba kembali. Hal serupa dapat terjadi ketika consumer sudah melakukan commit lokal, tetapi proses mati sebelum ACK diterima broker. Message ID `hazard_id:revision` membantu broker mengenali publish ulang selama dedup window 2 menit, tetapi perlindungan tersebut memiliki batas waktu.

Untuk menangani pengiriman ulang, setiap consumer menyimpan `message_id` pada inbox dengan constraint unik. Pencatatan inbox dan hasil olahan dilakukan dalam satu transaksi SQLite, kemudian consumer mengirim ACK. Jika pesan yang sama datang kembali, inbox menunjukkan bahwa hasilnya sudah diterapkan sehingga side effect lokal tidak diulang. Dashboard hanya menerima revisi yang lebih baru, sedangkan Notifier menggunakan identitas pesan yang unik untuk alert-nya.

Pesan dengan envelope tidak valid disimpan ke quarantine sebelum ACK, agar pesan rusak tidak terus diulang dan payload tetap tersedia untuk diperiksa. Jika penyimpanan gagal, consumer meminta pengiriman ulang sehingga pesan belum dianggap selesai.

Saat publish gagal, relay mencoba kembali dengan backoff 1 sampai 30 detik. Pending outbox dibatasi 10.000 pesan; jika batas ini tercapai, ingest tidak terus menambah antrean. Penghentian satu consumer karena itu tidak langsung menghalangi producer, selama broker masih dapat menyimpan event. Namun, kapasitas broker dan disk tetap menjadi batas: dengan DiscardNew, publish baru ditolak ketika stream penuh dan pesan tetap menunggu di outbox.

## 8.2. Alasan Keputusan dan Trade-off

Pemilihan mekanisme mempertimbangkan dua risiko: perubahan data tidak sempat dipublikasikan, dan consumer tidak tersedia saat event terbit. Outbox menangani kebutuhan publish yang persisten, sedangkan stream dan durable consumer menangani penyimpanan serta pembacaan event.

### 8.2.1 Outbox dan fan-out

Outbox dipilih untuk menangani celah antara penyimpanan data dan publish. Jika publish langsung dilakukan setelah commit, proses dapat mati ketika data sudah tersimpan tetapi pesan belum terkirim. Sebaliknya, publish sebelum commit dapat mengirim event dari transaksi yang kemudian gagal. Outbox mencatat kebutuhan publish dalam transaksi data yang sama, lalu relay mencobanya kembali. Konsekuensinya, ada jeda sebelum event sampai ke consumer dan kemungkinan publish berulang.

Jika kedua aplikasi berbagi satu durable, pesan akan dibagi sebagai pekerjaan untuk pembaca durable tersebut. Pola itu tidak memenuhi kebutuhan fan-out, yaitu event yang sama dapat diterima oleh setiap aplikasi. Karena itu, kedua consumer memakai pull consumer yang independen meskipun stream-nya sama.

### 8.2.2 Broker, pola baca, dan retention

Pull dipilih agar consumer meminta batch berikutnya setelah pengolahan batch sebelumnya selesai. Dibanding push, laju pengambilan lebih langsung dikendalikan aplikasi; ukuran batch dan batas ACK membatasi pesan yang sedang ditangani. Saat stream kosong, consumer tetap melakukan fetch dengan timeout.

LimitsPolicy mempertahankan event sampai batas retention, walaupun consumer telah memberikan ACK. Dengan DeliverAll, consumer baru dapat membaca event lama yang masih tersimpan. InterestPolicy juga mendukung fan-out, tetapi menghapus pesan setelah seluruh consumer yang berkepentingan memberikan ACK. Sementara itu, WorkQueuePolicy ditujukan untuk pembagian pekerjaan dengan satu pemroses per pesan [8]. LimitsPolicy dipilih agar penambahan consumer pada P5 dapat disertai replay, dengan biaya penyimpanan event selama masa retention.

Alternatif producer memanggil setiap consumer melalui HTTP membutuhkan daftar tujuan dan retry per tujuan di producer. NATS Core memberi fan-out sederhana, tetapi tidak menyimpan backlog durable untuk consumer yang mati. JetStream dipilih agar replay dan posisi baca dikelola broker. Kafka dan RabbitMQ juga dapat memenuhi kebutuhan ini; setup JetStream satu node dipilih karena fungsi stream dan durable sudah tersedia dalam executable NATS. Pertimbangan teknologi broker dijelaskan pada Bab 3.

## 8.3. Batasan yang Disadari

Jaminan idempotensi berlaku pada hasil olahan di database lokal. Seluruh alur belum dijamin exactly-once; panggilan ke layanan SMS eksternal, jika ditambahkan, berada di luar transaksi SQLite dan memerlukan penanganan idempotensi sendiri.

Consumer dapat mengejar backlog selama event yang dibutuhkannya masih tersedia di stream. Durable mempertahankan posisi baca, tetapi tidak memperpanjang retention. Jika consumer berhenti melewati batas retention, event yang sudah kedaluwarsa tidak dapat diambil kembali dari broker.

Konfigurasi memakai satu replica untuk deployment pada satu host, sehingga belum menyediakan failover antarmesin. Di sisi consumer, idempotensi bergantung pada riwayat inbox yang tersimpan di volume lokal. Jika volume tersebut dihapus, consumer kehilangan catatan pesan yang pernah diolah. Backup serta pemulihan volume belum

menjadi bagian dari pengujian M1. Retention broker dan keberlangsungan volume karena itu menjadi bagian dari batas jaminan pengiriman pada M1.

## 8.4. Bukti Penyelesaian

Pengujian P5 mengikuti posisi event sebelum, selama, dan setelah Dashboard berhenti. Setelah recovery diperiksa, pengujian menambahkan consumer ketiga untuk membuktikan producer tidak perlu diubah. Buktinya ditunjukkan melalui command berikut dan hasil trial integrasi [10].

### 8.4.1 Mencatat posisi dua durable consumer

Posisi awal dicatat agar event yang terbit selama gangguan dapat dibedakan dari event setelah recovery. Pengujian membaca endpoint monitoring JetStream melalui `wget` di dalam container NATS karena port monitoring tidak dibuka ke host. Query `streams=true&consumers=true` meminta informasi stream beserta consumer-nya, lalu `jq` memilih dua durable utama.

Pada output, `last_seq` adalah sequence terakhir stream, sedangkan `ack` menampilkan ACK floor consumer dalam sequence stream. ACK floor menunjukkan posisi yang sudah diakui secara berurutan. `pending` menghitung pesan yang belum dikirim, dan `inflight` menghitung pesan yang sudah dikirim tetapi belum di-ACK. Karena dua jenis antrean ini berbeda, pending nol saja belum membuktikan recovery selesai.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 308,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.1: Posisi awal stream dan dua durable utama sebelum Dashboard dihentikan.

Pada snapshot awal, Dashboard dan Notifier sama-sama memiliki ACK 308, sesuai sequence terakhir stream. Kesamaan posisi tersebut menunjukkan bahwa kedua consumer telah mengakui event hingga sequence terakhir stream.

### 8.4.2 Menghentikan satu consumer dan membaca backlog

Dashboard dihentikan melalui Compose, sementara producer dan Notifier dibiarkan berjalan. Jeda 15 detik memberi kesempatan bagi mock untuk menghasilkan event baru dan Aggregator untuk meneruskannya ke broker.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ sleep 15
```

Gambar 8.2: Menghentikan Dashboard tanpa menghentikan producer, broker, atau Notifier.

Command monitoring yang sama kemudian dijalankan ulang. Sequence stream naik dari 308 menjadi 311 dan ACK Notifier ikut mencapai 311. ACK Dashboard tetap 308 dengan tiga pesan pending. Jadi, event baru tetap tersimpan dan diolah Notifier saat Dashboard tidak aktif. Sequence 311 dicatat sebagai target yang harus dicapai Dashboard setelah kembali berjalan.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 311,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 3,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 311,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.3: Stream dan Notifier terus maju, sedangkan backlog Dashboard bertambah.

Dashboard dijalankan kembali, kemudian pengujian menunggu 35 detik sebelum membaca posisi ACK. Jeda ini juga memberi ruang untuk redelivery jika masih ada pesan yang dikirim sebelum consumer berhenti; konfigurasi AckWait adalah 30 detik.

```
$ docker compose start dashboard-consumer
 Container aat-report-revision-nats-1  Waiting
 Container aat-report-revision-nats-1  Healthy
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started

$ sleep 35
```

Gambar 8.4: Menjalankan kembali Dashboard dan menunggu pemrosesan backlog.

Pada pemeriksaan berikutnya, ACK Dashboard mencapai 318, melewati target 311 yang dicatat saat ia mati. Notifier juga mencapai 318, dan kedua consumer tidak mempunyai pesan pending maupun inflight pada snapshot tersebut. Hasil ini membuktikan Dashboard memproses event yang terbit selama gangguan, bukan hanya menerima event setelah startup.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 318,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.5: Dashboard sudah mengakui seluruh pesan hingga melewati target periode gangguan.

Inspeksi database lokal kemudian menunjukkan bahwa kedua consumer masing-masing memiliki 319 pesan pada inbox dan quarantine nol. Jumlahnya dapat lebih tinggi dari snapshot broker sebelumnya karena stream terus menerima event selama inspeksi. Inbox yang terpisah menunjukkan bahwa setiap aplikasi mencatat pengolahannya secara independen.

```
$ docker compose exec -T dashboard-consumer /dashboard-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}

$ docker compose exec -T notifier-consumer /notifier-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}
```

Gambar 8.6: Hasil olahan berada pada inbox lokal masing-masing consumer.

### 8.4.3 Menambahkan consumer tanpa mengubah producer

Penambahan consumer diuji pada sesi terpisah. Sebelum Third Consumer dijalankan, ID, waktu mulai, dan restart count Aggregator disimpan. Profile `demo` kemudian menambahkan container Third Consumer dengan durable `third-consumer-v1`, tanpa membangun atau menjalankan ulang Aggregator. Consumer ini memakai binary Notifier dengan durable dan volume miliknya sendiri.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-before.json
> docker compose --profile demo up -d --no-deps --wait third-consumer
> sleep 5
#0 building with "default" instance using docker driver

#1 [third-consumer internal] load build definition from Dockerfile
```

```
#15 DONE 0.1s
 Volume "aat-report-revision_third-consumer-data"  Creating
 Volume "aat-report-revision_third-consumer-data"  Created
 Container aat-report-revision-third-consumer-1  Creating
 Container aat-report-revision-third-consumer-1  Created
 Container aat-report-revision-third-consumer-1  Starting
 Container aat-report-revision-third-consumer-1  Started
 Container aat-report-revision-third-consumer-1  Waiting
 Container aat-report-revision-third-consumer-1  Healthy
```

Gambar 8.7: Cuplikan awal dan akhir command startup Third Consumer melalui profil demo; bagian tengah log build tidak ditampilkan.

Pada sesi tersebut, snapshot monitoring menunjukkan durable baru telah mencapai ACK 192, sama dengan posisi stream dan kedua durable lainnya. Karena consumer baru memakai DeliverAll, pesan yang masih tersimpan sebelum ia dibuat juga tersedia untuk dibaca; producer tidak perlu mengirim ulang secara khusus kepadanya.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 192,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "notifier-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "third-consumer-v1",
      "ack": 192,
      "pending": 0
    }
  ]
}
```

Gambar 8.8: Durable baru membaca stream yang sama sampai posisi ACK yang ditampilkan broker.

Perbandingan kondisi Aggregator sebelum dan sesudah menghasilkan file yang identik. Inspeksi Third Consumer juga menunjukkan inbox 193 dengan quarantine nol. Jumlah inbox diambil setelah snapshot broker, ketika stream masih menerima event. Dengan demikian, penambahan consumer berhasil tanpa perubahan container producer, dan consumer baru berhasil menyimpan hasil pengolahannya.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-after.json
> diff -s /tmp/report-session/producer-before.json \
>   /tmp/report-session/producer-after.json
> docker compose --profile demo exec -T third-consumer \
>   /notifier-consumer inspect | jq '{inbox_count, quarantine_count}'
>
Files /tmp/report-session/producer-before.json and /tmp/report-
session/producer-after.json are identical
{
  "inbox_count": 193,
  "quarantine_count": 0
}
```

Gambar 8.9: Producer tidak berubah dan consumer tambahan memiliki inbox sendiri.

Hubungan hasil pengujian dengan kriteria P5 dirangkum pada tabel berikut.

| Kriteria | Bukti |
|---|---|
| P5.1 | Publish relay ke subject broker pada kode dan kenaikan sequence ketika Dashboard mati. |
| P5.2 | Posisi durable terpisah pada Gambar 8.1 dan inbox masing-masing pada Gambar 8.6. |
| P5.3 | Sequence 311 sebagai target gangguan, lalu ACK Dashboard 318 pada Gambar 8.3 dan 8.5. |
| P5.4 | Startup Third Consumer, posisi ACK, dan kondisi producer pada Gambar 8.7–8.9. |
| P5.5 | Broker, pola fan-out, semantik at-least-once, dan konsekuensi idempotensi consumer pada Bagian 8.5. |

## 8.5. Jawaban atas Butir “Di laporan”

Broker yang dipilih adalah NATS JetStream dengan pola fan-out melalui stream `HAZARD_EVENTS`. Setiap aplikasi memakai durable pull consumer sendiri, sehingga seluruh aplikasi dapat membaca event yang sama tanpa daftar tujuan pada producer. LimitsPolicy dan DeliverAll memungkinkan consumer yang pulih atau baru ditambahkan membaca event yang masih berada dalam retention.

Semantik pengiriman adalah **at-least-once**: kegagalan pencatatan status publish atau ACK dapat menyebabkan pesan dikirim ulang. Karena itu, consumer harus idempoten. Kelompok menerapkannya dengan unique `message_id` pada inbox dan transaksi SQLite yang mencatat inbox bersama hasil olahan sebelum ACK. Dedup broker membantu selama window 2 menit, tetapi inbox tetap diperlukan untuk redelivery di luar window tersebut. Jaminan ini berlaku pada hasil lokal; panggilan notifikasi eksternal kelak memerlukan penanganan idempotensi tersendiri.

# KESIMPULAN

Implementasi M1 menghubungkan mock BMKG dan PVMBG melalui Aggregator, lalu menyediakan hasilnya kepada client dan consumer dengan kontrak yang terpisah. Pada profil data awal yang diuji, pemetaan menghasilkan HazardEvent yang valid dan menerima tambahan atribut PVMBG tanpa restart atau migrasi tabel. Client API juga membatasi field sesuai identitas, termasuk menolak permintaan data raw dari Media.

Pemisahan jalur polling dan baca menjaga layanan ketika PVMBG mengalami gangguan. Saat outage, API tetap mengembalikan data terakhir dengan penanda stale, sementara BMKG tetap diperbarui. Pada load test dengan 60 worker dan delay PVMBG 3 detik, p95 BMKG mencapai 124,52 ms, throughput 980,26 respons sukses per detik, dan error selain 429 sebesar 0,034%. Sampel TCP menunjukkan sedikitnya 50 koneksi selama 89,00 detik. Hasil tersebut memenuhi ambang P2 pada profil beban yang diuji.

Pada jalur event, Dashboard dapat dihentikan atau dibangun ulang ketika producer dan Notifier tetap berjalan. Setelah pulih, Dashboard melanjutkan pembacaan event yang tersimpan di broker. Third Consumer juga berhasil ditambahkan tanpa perubahan producer. Hasil ini menunjukkan bahwa container terpisah, durable independen, dan storage privat mendukung operasi serta pemulihan per consumer.

Batas utama yang masih perlu ditangani adalah pertumbuhan data dan deployment satu instance. Sebelum meningkatkan volume, sistem perlu mengurangi biaya full snapshot, menguji ulang kapasitas, serta menetapkan pembersihan history, inbox, dan outbox. Jika jumlah instance ditambah pada milestone berikutnya, polling, relay, dan state service juga membutuhkan koordinasi. Di sisi keamanan, sinkronisasi waktu, TLS, pemisahan credential broker, serta rotasi key perlu disiapkan untuk penggunaan di luar demo lokal.

# PEMBAGIAN TUGAS

| Anggota | Perancangan | Implementasi | Penulisan laporan |
|---|---|---|---|
| M Hazim R Prajoda 13523009 | Kontrak sumber, aturan pemetaan, Canonical Store | Mock BMKG/PVMBG, Aggregator, polling dan penyimpanan. Review implementasi Bimo dan Refki. | P1 dan aspek flexible storage P4 |
| Adhimas Aryo Bimo 13523052 | Identitas client, scope, token, isolasi request | Auth/Token Service, Client API, ketahanan dan load test. Review implementasi Hazim dan Refki. | P2, P3, serta pengukuran API |
| Refki Alfarizi 13523002 | Event fan-out, durable consumer, jaringan dan integrasi | JetStream, consumer, orchestration dan trial integrasi. Review implementasi Hazim dan Bimo. | P5, container P4, penyusunan dan penggabungan laporan |

Tabel 10.1: Kontribusi anggota pada perancangan, implementasi, dan penulisan laporan.

# DAFTAR REFERENSI


1. IF4031. (2026). *Tugas Besar IF4031: Arsitektur Aplikasi Terdistribusi, Milestone 1: Sistem Terdistribusi Dasar*. Dokumen spesifikasi mata kuliah.
2. IF4031. (2026). *Tugas Besar IF4031: Arsitektur Aplikasi Terdistribusi*. Dokumen utama tugas besar.
3. Go Authors. *Package net/http*. `https://pkg.go.dev/net/http`.
4. Go Authors. *Package encoding/json*. `https://pkg.go.dev/encoding/json`.
5. PostgreSQL Global Development Group. *JSON Types*. `https://www.postgresql.org/docs/current/datatype-json.html`.
6. NATS.io. *JetStream*. `https://docs.nats.io/nats-concepts/jetstream`.
7. NATS.io. *JetStream Consumers*. `https://docs.nats.io/nats-concepts/jetstream/consumers`.
8. NATS.io. *JetStream Retention Policies*. `https://docs.nats.io/learn/jetstream/retention-policies`.
9. Docker. *Docker Compose documentation*. `https://docs.docker.com/compose/`.
10. DikejarDeadline. (2026). *Bukti pengujian Milestone 1*. Berkas pada `docs/evidence/` dan `docs/report/evidence/` mencakup trial 5 Oktober 2026 serta tangkapan command 7 Oktober 2026. Konfigurasi, hasil load/TCP, dan command dicatat dalam `environment.json`, `load.json`, `assessment.json`, dan `terminal-commands.json`.
11. Jones, M., Bradley, J., dan Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. `https://www.rfc-editor.org/rfc/rfc7519`.
12. RabbitMQ. *Exchanges*. `https://www.rabbitmq.com/docs/exchanges`.
13. Apache Kafka. *Introduction*. `https://kafka.apache.org/intro/`.

# FORMAT DEKLARASI PENGGUNAAN AI
## SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya/~~kami*:

| | | |
|---|---|---|
| Nama/NIM | : | Refki Alfarizi / 13523002<br>M Hazim R Prajoda / 13523009<br>Adhimas Aryo Bimo / 13523052 |
| Fakultas/Sekolah | : | Sekolah Teknik Elektro dan Informatika (STEI), ITB |
| Kode Mata Kuliah | : | IF4031 |
| Nama Mata Kuliah | : | Arsitektur Aplikasi Terdistribusi |
| Judul Tugas/Proposal | : | Tugas Besar IF4031 - milestone 1: Sistem Terdistribusi Dasar |

Menyatakan bahwa ~~saya/~~kami* menggunakan/~~tidak menggunakan*~~ AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

J yang digunakan adalah:

| Lingkup Pekerjaan | Digunakan? | Tingkat Penggunaan |
|---|---|---|
| **Pemeriksaan Ejaan:** menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. | Ya | Terbatas: penyuntingan bahasa laporan. |
| **Pembuatan Teks:** menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. | Ya | Draf laporan dan kode Go; bantuan tes/debugging. |
| **Bantuan Diskusi dalam Penyusunan Konten:** menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya. | Ya | Diskusi desain, analisis kode, dan verifikasi. |
| **Pembuatan Gambar, Video, dan/atau Grafik:** menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. | Ya | Draf diagram Mermaid/TikZ; bukan foto/video. |
| **Lainnya, Jelaskan:** | Tidak | |

~~Saya/~~Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 7 September 2026

| | | |
|---|---|---|
| Refki Alfarizi | M Hazim R Prajoda | Adhimas Aryo Bimo |
| 13523002 | 13523009 | 13523052 |